A POWER-USER'S GUIDE

Maximum Autonomy

Every setting, flag, config file, and token choice for hands-off AI agents, tool by tool

For people ready to let agents run • October 2026 edition • Print edition · US Letter

GUIDES FOR EVERYTHING

Maximum Autonomy

Every setting, flag, config file, and token choice for hands-off AI agents, tool by tool

This guide is for one goal: letting your AI agents work **hands-off**, with as few approval prompts as possible, across Claude Code, the Claude apps, Cursor, VS Code and Copilot, Codex, Gemini CLI, GitHub, MCP servers, n8n, and Notion.

You've decided you accept the risk, so this guide doesn't argue with you. It gives you the exact switches. It also shows the one habit that makes maximum autonomy sustainable: **contain the agent instead of restricting it**. Give it a box where mistakes are cheap, then remove the guardrails inside that box. Each chapter ends with what *still* prompts, no matter what you set.

HOW TO READ THIS MANUAL

Tip — a shortcut or good habit.

Note — useful background.

Warning — easy to get wrong.

Danger — can lose data, money, or access.

`$ command` — text you type, exactly as shown.

Settings › Privacy — a menu path to click.

New words appear in **bold** the first time and are collected in the Glossary at the back.

Settings, flags, and menu paths were checked against each vendor's official documentation in October 2026. These products change monthly, and safety prompts in particular are added and renamed often. If a setting name doesn't match what you see, check the vendor's current docs. You are responsible for what your agents do with the access you grant.

Contents

01 **The Autonomy Playbook** 5

1.1 Four levels of autonomy 5
1.2 Contain, don't restrict 5
1.3 Quick-start: maximum autonomy at a glance 6

02 **Build a Safe Playground** 7

2.1 Option 1: a dev container (recommended for coding agents) 7
2.2 Option 2: a dedicated machine or cloud box 8
2.3 Option 3: a separate user account on your computer 8
2.4 Accounts, tokens, and money 8

03 **Claude Code (Terminal)** 9

3.1 Level 3: auto mode by default 9
3.2 Level 4: bypass permissions 10
3.3 Fully unattended runs 11
3.4 A complete maximum-autonomy settings file 11
3.5 Reduce prompts without bypass 12
3.6 What still prompts in Claude Code, no matter what 13

04 **Claude Desktop, Cowork, and claude.ai** 14

4.1 The Code tab in Claude Desktop 14
4.2 Cowork 15
4.3 Connectors in claude.ai and the Claude apps 15
4.4 Claude in Chrome 15

05 **Cursor** 16

5.1 The editor's agent 16
5.2 The Cursor CLI 17

06 **VS Code and GitHub Copilot** 18

6.1 Copilot agent in VS Code 18
6.2 Copilot CLI 19
6.3 Copilot coding agent (on GitHub) 19

07 **Codex, Gemini CLI, and Builders** 20

7.1 OpenAI Codex 20
7.2 Gemini CLI 20
7.3 Browser app builders 21

08 **GitHub: Tokens and Repository Settings** 22

8.1 Authenticate agents broadly 22

8.2 Repository settings that remove friction 23
8.3 What still prompts on GitHub 24

09 MCP Servers and Connectors 25
9.1 Give servers enough power 25

10 n8n 26
10.1 Let AI agents build and run your workflows 26
10.2 Self-hosted: unlock everything 26

11 Notion 28
11.1 Notion MCP (simplest, full personal access) 28
11.2 An API connection with every capability 28
11.3 Notion AI agents 28

12 What Always Asks: The Master List 30

13 Claude Code Permission Modes and Settings Reference 31
13.1 The six permission modes 32
13.2 Choosing a mode 32
13.3 Every settings key, A–Z 33

14 Autonomy Recipes 43
14.1 Golden rules of autonomy 43

15 Incident Response: When an Agent Goes Wrong 44

16 Codex config.toml Reference 46
16.1 Command-line autonomy flags (current release) 46
16.2 config.toml keys (292) 47
16.3 requirements.toml keys (174) 61

17 Gemini CLI settings.json Reference 71

Your Autonomy Setup Checklist 81

Glossary 82

CHAPTER 1

The Autonomy Playbook

IN THIS CHAPTER

- The four levels of agent autonomy
- “Contain, don’t restrict”: the principle behind this guide
- The quick-start setup for each tool

1.1 Four levels of autonomy

Every agent tool offers some version of these four levels:

LEVEL	WHAT HAPPENS	EXAMPLES
1. Ask	The agent asks before most actions	Claude Code *Manual*, Cursor *Allowlist* with an empty list
2. Allowlist	Pre-approved actions run; everything else asks	Allow rules, terminal allowlists
3. Classifier	A second AI reviews each action and blocks only risky ones	Claude Code *Auto*, Cursor *Auto-review*
4. Bypass	Everything runs. No checks except a few hard-coded ones	`--dangerously-skip-permissions`, Cursor *Run Everything*, Codex `--yolo`

Level 3 gets you about 95% of the hands-off feel while still catching disasters. Level 4 is true maximum autonomy. This guide gives you both, so you can choose per tool and per project.

1.2 Contain, don’t restrict

Prompts exist to stop an agent from damaging things that matter. You can get the same protection a better way: put the agent somewhere where **nothing it can reach matters too much**. Then you can safely turn the prompts off.

- **A disposable machine:** a container, a virtual machine, a cloud dev box, or a spare computer.

- **Separate accounts:** a dedicated GitHub account or bot for the agent, a separate Notion workspace or teamspace, a separate n8n instance.
- **Narrow-but-complete tokens:** full power *inside* the agent's projects, nothing outside them.
- **Budgets:** spending caps on every API and cloud account.
- **Backups and Git:** everything committed, everything restorable.

Chapter 2 shows how to set this up in an afternoon.

> ☆ IMPORTANT
>
> The single biggest real-world risk of hands-off agents is **prompt injection**: instructions hidden in a web page, email, issue, or document that the agent reads and obeys. Containment limits the damage it can do. Bypass settings remove the last line of defense against it.

1.3 Quick-start: maximum autonomy at a glance

TOOL	HANDS-OFF WITH SAFETY REVIEW	FULL BYPASS
Claude Code (CLI)	`"defaultMode": "auto"` in `~/.claude/settings.json`	`claude --dangerously-skip-permissions`
Claude Desktop, Code tab	Mode selector, then **Auto**	Settings › Claude Code › **Allow bypass permissions mode**, then **Bypass permissions**
Claude Cowork	Mode selector, then **Automatically approve**	(no separate bypass)
claude.ai connectors	Each tool set to **Always allow**	(same)
Cursor	Approvals & Execution: **Auto-review**	Approvals & Execution: **Run Everything**
VS Code + Copilot	**Assisted permissions** (experimental)	**Allow all** / **Autopilot**, `chat.tools.global.autoApprove`
Copilot CLI	Allow rules	`copilot --yolo` (alias of `--allow-all`)
OpenAI Codex	`--sandbox workspace-write` or `--approve-for-me`	`codex --yolo`, or `approval_policy = "never"` + `sandbox_mode = "danger-full-access"`
Gemini CLI	`--approval-mode=auto_edit`	`gemini --yolo`
GitHub	Bot account + fine-grained token	Broad token + Actions write permissions + auto-merge
n8n	Instance-level MCP + API key	Plus `NODES_EXCLUDE="[]"` on self-hosted
Notion	Notion MCP (OAuth)	Internal connection with all capabilities, shared at the top level

CHAPTER 2

Build a Safe Playground

Fifteen minutes of setup here lets you run everything else in this guide at full throttle.

2.1 Option 1: a dev container (recommended for coding agents)

A **dev container** is a Docker container configured as a complete development environment. Anthropic publishes a reference dev container for Claude Code that runs as a **non-root user** with a network firewall, which is exactly the setup bypass mode is designed for.

RUN AGENTS IN A DEV CONTAINER

1. **Install Docker Desktop** (docker.com) and **VS Code** with the **Dev Containers** extension.
2. **Add a `.devcontainer/` folder** to your project. Copy Anthropic's reference configuration from the Claude Code repository (`anthropics/claude-code`, folder `.devcontainer`), or ask Claude Code: "set up a dev container for this project based on the official Claude Code devcontainer."
3. **Open the project in the container.**
   Command Palette, then **Dev Containers: Reopen in Container**.
4. **Run your agent inside it** in bypass mode. If it goes wrong, rebuild the container; your host machine is untouched.

> **NOTE**
>
> On Linux and macOS, Claude Code refuses `--dangerously-skip-permissions` when run as **root** or with `sudo`, except inside a recognized sandbox. Dev containers and a normal user account both satisfy this.

2.2 Option 2: a dedicated machine or cloud box

A spare laptop, a small cloud VM, or a cloud dev environment (GitHub Codespaces, a cloud VM, or Claude Code's own cloud sessions at claude.ai/code) all work. Give it its own accounts and tokens, and nothing personal.

2.3 Option 3: a separate user account on your computer

Create a second user account on your Mac or PC just for agents. Your personal files, browser logins, and SSH keys stay out of reach.

2.4 Accounts, tokens, and money

- [ ] A separate GitHub account (or GitHub App) for agents, added as a collaborator only on the repos they work on
- [ ] Fine-grained tokens scoped to those repos, with long-enough expiry
- [ ] A separate Notion teamspace or workspace for agent-writable content
- [ ] Hard monthly spend limits on Anthropic, OpenAI, Google, and cloud accounts
- [ ] Automatic backups (Time Machine, File History, or cloud backups) of anything the agent can write
- [ ] Everything in Git, pushed often, with `main` protected (Chapter 8)

CHAPTER 3

Claude Code (Terminal)

Claude Code has the most complete autonomy controls of any tool. Two settings files matter:

- `~/.claude/settings.json`: **user** settings. Your machine-wide defaults.
- `.claude/settings.json` (project) and `.claude/settings.local.json` (project, just you).

> ⚠ WARNING
>
> `"defaultMode": "auto"` and `"defaultMode": "bypassPermissions"` are **ignored** in project files (`.claude/settings.json` and `.claude/settings.local.json`); this stops a cloned repository from switching your agent to bypass. Put them in `~/.claude/settings.json`, in managed settings, in a `--settings` file, or pass the flag.

3.1 Level 3: auto mode by default

Recent versions of Claude Code already start interactive sessions in **auto mode**, where a classifier model reviews risky actions instead of asking you. To make it explicit and permanent:

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "auto"
  }
}
```

Then reduce the classifier's false alarms:

- **Teach it your infrastructure.** Run `/auto-mode-setup` to draft `autoMode.environment` entries (trusted repos, buckets, domains, services) from your project and recent sessions, and save them.

- **Approve standing patterns.** Tell it in plain English with specifics ("you may force-push to branches starting with `agent/`"), or add rules under `autoMode.allow`.
- **Check the rules:** `claude auto-mode defaults` prints the built-in block/allow lists; `claude auto-mode config` shows your effective config.

> **NOTE**
> If the classifier blocks 3 actions in a row or 20 in total, auto mode pauses and asks you. These thresholds can't be changed. In `claude -p` runs with no prompt available, the blocked action is skipped and Claude keeps working.

3.2 Level 4: bypass permissions

One session:

TERMINAL

```
claude --dangerously-skip-permissions
```

(Equivalent: `claude --permission-mode bypassPermissions`.) The first time, you accept a warning; Claude Code then records `"skipDangerousModePermissionPrompt": true` in `~/.claude/settings.json` so it doesn't ask again.

Every session, by default:

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "bypassPermissions"
  },
  "skipDangerousModePermissionPrompt": true
}
```

Start in another mode but keep bypass one keypress away: `claude --allow-dangerously-skip-permissions` adds bypass to the Shift + Tab cycle without starting in it.

A shell alias for convenience:

~/.ZSHRC OR ~/.BASHRC

```
alias yolo='claude --dangerously-skip-permissions'
```

3.3 Fully unattended runs

For scripts, scheduled jobs, and CI, use print mode:

TERMINAL

```
claude -p "Fix the failing tests and commit the result" \
  --dangerously-skip-permissions \
  --permission-prompts none \
  --max-turns 60 \
  --max-budget-usd 10
```

- `--permission-prompts none` denies anything that would still need a human, instead of waiting forever.
- `--max-turns` and `--max-budget-usd` are your seat belts.
- `--output-format json` gives machine-readable results.
- `claude --bg "…"` starts a **background agent** that keeps running after you close the terminal; manage it with `claude agents`. Background sessions honor bypass only if your acceptance is recorded in `~/.claude/settings.json`.
- In CI without bypass, use `--permission-mode dontAsk` with an exact `--allowedTools` list.

3.4 A complete maximum-autonomy settings file

~/.CLAUDE/SETTINGS.JSON

```
{
  "permissions": {
    "defaultMode": "bypassPermissions",
    "allow": [
      "Bash",
      "Edit",
      "Write",
      "WebFetch",
      "WebSearch",
      "mcp__github__*",
      "mcp__notion__*"
    ],
    "additionalDirectories": ["~/projects"]
  },
  "skipDangerousModePermissionPrompt": true,
  "enableAllProjectMcpServers": true,
  "env": {
    "CLAUDE_CODE_DISABLE_DANGEROUS_RM_TIMEOUT": "1"
  }
}
```

What each line buys you:

SETTING	EFFECT
`defaultMode: bypassPermissions`	Every session starts with no prompts
`allow` **rules**	Still useful when you drop into auto or manual mode: these tools stop asking. (Allow rules have no effect *in* bypass, which already allows everything; and broad rules like `Bash` are dropped while in auto mode.)
`mcp__server__*`	Approves every tool from that MCP server. Allow globs must name the server; `mcp__*` alone is ignored
`additionalDirectories`	Lets Claude read and edit outside the folder you started in without asking
`enableAllProjectMcpServers`	Auto-approves servers listed in projects' `.mcp.json` files
`CLAUDE_CODE_DISABLE_DANGEROUS_RM_TIMEOUT`	Removes the 2-minute countdown on critical deletions (the prompt itself stays)

> **TIP**
> Workspace trust: the first time you open a folder, Claude Code asks whether you trust it. Answer once per repository and it remembers. For automation, `-p` runs don't show the dialog, though project allow rules aren't applied there until a folder is trusted.

3.5 Reduce prompts without bypass

If you'd rather stay in Manual or Accept-edits mode for some projects:

- `/fewer-permission-prompts` scans your history and proposes an allowlist.
- `/sandbox`, then **auto-allow**, runs shell commands inside an OS-level sandbox *without prompting* (macOS, Linux, WSL2).
- Answer prompts with **Yes, and don't ask again** to save rules automatically.

3.6 What still prompts in Claude Code, no matter what

STILL ASKS, EVEN IN BYPASS	WHY / WORKAROUND
Tools matching an explicit ask rule	Remove the ask rule
`rm` / `rmdir` **targeting a critical path:** `/`**, top-level folders, your home folder, the working directory or its parents**	A deliberate circuit breaker. In bypass it asks (with a 2-minute countdown that auto-denies). Use literal subfolder paths instead
`AskUserQuestion` **and MCP tools marked requiresUserInteraction**	By design; they need an answer from a person
Connector tools your organization set to "ask"	Admin policy
Reads outside working directories if `blockReadsOutsideWorkingDirectories` **is on**	Leave it off, or add directories
Some cross-session messaging approvals	Only matters with multiple agent sessions messaging each other
`Remove-Item` **on system paths or bare wildcards (PowerShell)**	Denied outright in every mode
Cloud sessions (claude.ai/code)	Bypass isn't offered; **Auto** is the maximum

CHAPTER 4

Claude Desktop, Cowork, and claude.ai

4.1 The Code tab in Claude Desktop

The Code tab uses the same engine and settings files as the CLI.

ENABLE BYPASS IN THE DESKTOP APP

1. **Open Settings** in Claude Desktop and go to **Claude Code**.
2. **Turn on "Allow bypass permissions mode."** On Pro and Max plans this is a personal toggle; on Team and Enterprise, your organization's policy controls it.
3. **Choose the mode.**
   In a session, open the mode selector next to the send button (or press `Cmd` + `Shift` + `M`) and pick **Bypass permissions**, or **Auto** for classifier-reviewed autonomy.
4. **It sticks per folder.**
   The desktop app remembers the mode you pick for each project folder (except Plan). A `defaultMode` in `~/.claude/settings.json` sets the starting point for new folders.

Still asks in the desktop app: the Claude Code list from Chapter 3, plus **safety classifiers on actions Claude takes on external websites** in the built-in browser (any mode), and destructive desktop actions such as deleting a session. On websites, Claude won't purchase things, create accounts, or solve CAPTCHAs without you. Answer a site's first action card with **Always allow** to stop repeat prompts for that site.

4.2 Cowork

Cowork (a tab in Claude Desktop) does longer agentic work on your files, in your browser, and through connectors.

- In the Cowork mode selector, choose **Automatically approve**. On Team and Enterprise, an admin must leave **Allow "Automatically approve" mode** on (it's on by default) in `Organization settings` › `Cowork` › `Permissions`.
- For connector **write** tools to stop asking every task, your admin must also turn on **Allow "Always allow" for connector tools** (off by default on Team and Enterprise). Then choose **Allow for all tasks** in the approval dialog.

4.3 Connectors in claude.ai and the Claude apps

Each connector's tools can be set to **Always allow**, **Needs approval**, or **Blocked**.

MAKE CONNECTORS HANDS-OFF

1. **Open** `Settings` › `Connectors` (or **Customize**, then **Connectors**, in the desktop app).
2. **Click a connector**, then **Tool permissions**.
3. **Set each category or tool to Always allow.**
   Do this for read and write tools you want the agent to use freely.
4. **Repeat for each connector.**

> **NOTE**
> Custom connectors' tools usually lack "read-only" labels, so they default to asking. Set them to **Always allow** explicitly. Organization owners can cap what members may allow, and that cap can't be overridden by individuals.

4.4 Claude in Chrome

The Chrome extension asks per site the first time Claude acts there. Choose **Always allow** for sites you want it to operate on freely. It will still stop for purchases, account creation, and CAPTCHAs, and its safety classifier can still raise a prompt on risky page actions.

CHAPTER 5

Cursor

5.1 The editor's agent

TURN ON RUN EVERYTHING

1. **Open settings:**
   `Settings` › `Agents` › `Approvals & Execution`.
2. **Set the Run Mode to Run Everything.**
   Every tool call, including shell commands, MCP tools, and web fetches, runs automatically, with no sandbox and no classifier.
3. **Read Access** is hidden in Run Everything (nothing is checked). In other modes, set it to **System** to allow reads outside the workspace without asking.

Prefer a safety net? **Auto-review** runs allowlisted calls immediately, sandboxes shell commands where it can, and sends the rest to a classifier. Tune it in plain English in `~/.cursor/permissions.json`:

~/.CURSOR/PERMISSIONS.JSON

```
{
  "autoRun": {
    "allow_instructions": [
      "Any git command in this project, including push to branches starting with agent/.",
      "Installing packages with npm, pnpm, or pip."
    ],
    "block_instructions": []
  }
}
```

The sandbox's network access and extra writable paths live in `~/.cursor/sandbox.json`.

5.2 The Cursor CLI

The command-line agent reads permissions from `~/.cursor/cli-config.json` (global) or `.cursor/cli.json` (project):

~/.CURSOR/CLI-CONFIG.JSON

```
{
  "permissions": {
    "allow": ["Shell(*)", "Read(**)", "Write(**)"],
    "deny": []
  }
}
```

For one-off unattended runs, the CLI's `--force` option runs commands without prompting.

Still asks in Cursor: on a team plan, admin policies and Cursor's built-in security rules sit on top of your local files and can't be weakened by them. In Auto-review or Allowlist modes, commands that need to leave the sandbox ask first, as do writes to protected paths like `.git/config`, `.git/hooks`, `.vscode`, and `.cursorignore`.

CHAPTER 6

VS Code and GitHub Copilot

6.1 Copilot agent in VS Code

LEVEL	HOW
Allow all	Choose **Allow all** in the chat session's permission picker: "runs all tool calls without confirmation"
Autopilot	Choose **Autopilot** in the mode picker: auto-approves tools, retries errors, and answers blocking questions itself
Advanced Autopilot (preview)	`"chat.autopilot.advanced.enabled": true`, where a separate model decides when the task is complete
Every workspace	`"chat.tools.global.autoApprove": true`
This session only	Type `/yolo` (or `/autoApprove`); undo with `/disableYolo`

VS CODE SETTINGS.JSON

```
{
  "chat.tools.global.autoApprove": true,
  "chat.autopilot.advanced.enabled": true
}
```

Finer-grained alternatives: `chat.tools.terminal.autoApprove` (terminal command rules) and `chat.tools.urls.autoApprove` (web addresses). Terminal writes detected *outside* the workspace still ask by default; that's governed by `chat.tools.terminal.blockDetectedFileWrites`.

Running the **Claude agent harness inside VS Code**? Its equivalent of bypass is `"github.copilot.chat.claudeAgent.allowDangerouslySkipPermissions": true`. The **Claude Code extension** has its own **Allow dangerously skip permissions** toggle in its settings, after which **Bypass permissions** appears in its mode indicator.

Still asks in VS Code: tools or terminal rules you've set to `false`; organization (device-management) policies; and, unless you change `blockDetectedFileWrites`, terminal writes it detects outside the workspace. An enabled sandbox still limits file and network access even in

Allow all.

6.2 Copilot CLI

TERMINAL

```
copilot --yolo
```

`--yolo` is an alias for `--allow-all`, which combines `--allow-all-tools`, `--allow-all-paths`, and `--allow-all-urls`. Inside a session, `/yolo` or `/allow-all` does the same; `/reset-allowed-tools` revokes it. Business and Enterprise admins can block these.

6.3 Copilot coding agent (on GitHub)

Assign an issue to Copilot and it works in its own GitHub Actions environment, then opens a pull request. It already runs without prompts. Widen what it can reach in the repository's Settings › Copilot › Coding agent: add a **Custom allowlist** for its firewall, or turn the firewall off entirely (maximum reach, maximum exfiltration risk), and add MCP servers.

CHAPTER 7

Codex, Gemini CLI, and Builders

7.1 OpenAI Codex

GOAL	HOW
Fewer prompts, sandboxed	`codex --sandbox workspace-write` (agent edits inside the workspace), or `codex --approve-for-me` (workspace-write sandbox plus automatic review of approval requests). The old `--full-auto` flag is gone from current releases.
No prompts, no sandbox	`codex --yolo` (alias of `--dangerously-bypass-approvals-and-sandbox`)
Make it the default	Put the two keys below in `~/.codex/config.toml`

~/.CODEX/CONFIG.TOML

```
approval_policy = "never"
sandbox_mode = "danger-full-access"
```

In the Codex app and IDE extension, choose **Full Access** from the permissions menu.

7.2 Gemini CLI

TERMINAL

```
gemini --yolo
# or
gemini --approval-mode=yolo
```

YOLO mode can only be turned on from the command line, so use an alias (`alias gem='gemini --yolo'`). The settings file `~/.gemini/settings.json` supports `general.defaultApprovalMode` of `"default"`, `"auto_edit"` (auto-approve file edits), or `"plan"`. YOLO also switches on Gemini's

sandbox by default, which doesn't add prompts. Leave it on as free protection.

Still asks: if `security.disableYoloMode` or the admin setting `admin.secureModeEnabled` is true, YOLO is refused. Untrusted folders (folder trust) restrict what runs until you trust them.

7.3 Browser app builders

TOOL	MAX-AUTONOMY SETTING
Replit Agent	In the Agent's tools panel, set **Autonomy Level** to **Max** for long, unsupervised runs (labeled beta)
Lovable, Bolt, v0, Base44	They act without per-step approvals already. Give them a connected GitHub repo and backend, and let them run; review via version history

CHAPTER 8

GitHub: Tokens and Repository Settings

GitHub is where agents push code, open pull requests, and trigger automation. Here's how to give them full power with minimal friction.

8.1 Authenticate agents broadly

The GitHub CLI (used by Claude Code, Codex, and others):

TERMINAL

```
gh auth login
gh auth refresh -h github.com -s repo,workflow,read:org,gist,delete_repo
```

The `workflow` scope lets agents edit `.github/workflows/` files; without it, pushes that touch workflows are rejected. `delete_repo` is only needed if you want agents to delete repositories.

A fine-grained personal access token (for MCP servers, n8n, and scripts): go to `Settings` › `Developer settings` › `Personal access tokens` › `Fine-grained tokens`, then **Generate new token**:

OPTION	MAXIMUM-AUTONOMY CHOICE
Resource owner	Your account (or the organization the agent works in)
Expiration	As long as you're comfortable with, or custom
Repository access	**All repositories** (or every repo the agent should touch)
Contents	Read and write
Pull requests	Read and write
Issues	Read and write
Workflows	Read and write
Actions	Read and write
Administration	Read and write (only if agents create repos or change settings)
Metadata	Read (required)

Classic tokens are simpler but blunter: the `repo` and `workflow` scopes cover most agent work.

For teams: install the **Claude GitHub App** (run `/install-github-app` in Claude Code) on **All repositories** so `@claude` works everywhere.

8.2 Repository settings that remove friction

SETTING	WHERE	CHOOSE
Workflow permissions	Settings › Actions › General	**Read and write permissions**
Let Actions open and approve PRs	Same page	Tick **Allow GitHub Actions to create and approve pull requests**
Auto-merge	Settings › General, under Pull Requests	Tick **Allow auto-merge**; agents can then run `gh pr merge --auto --squash`
Branch rules	Settings › Rules › Rulesets	Remove required reviews, or add the agent's account or app to the ruleset's **Bypass list**
Delete merged branches	Settings › General	Tick **Automatically delete head branches**

> **TIP**
>
> A powerful, still-sane setup: leave `main` protected by status checks only (tests must pass), put your agent's account on nothing else, and turn on auto-merge. Agents ship on their own, but only green code lands.

8.3 What still prompts on GitHub

- **Sudo mode:** sensitive actions in the web interface (deleting repos, changing security settings) re-ask for your password or 2FA. Tokens and the API aren't affected.
- **Push protection:** pushes containing detected secrets are blocked until bypassed for that push.
- **Claude's auto mode** blocks merging pull requests no human approved, and force pushes, by default. Use bypass mode, or grant a specific standing approval, if you want agents to do that.
- **Organization policies** (SSO token authorization, required reviews, Copilot policies) override your personal choices.

CHAPTER 9

MCP Servers and Connectors

Every MCP tool call can trigger an approval prompt. Here's how to approve them up front in each host.

HOST	AUTO-APPROVE MCP TOOLS
Claude Code	Allow rules: `"mcp__github__*"`, `"mcp__notion__*"`, one per server (a bare `mcp__*` is ignored). Or bypass mode. `"enableAllProjectMcpServers": true` skips approving `.mcp.json` servers
Claude apps	Connector, then **Tool permissions**, then **Always allow**
Cursor	**Run Everything** covers MCP calls; Auto-review sends them to the classifier
VS Code	**Allow all**, Autopilot, or `chat.tools.global.autoApprove`
Codex	`approval_policy = "never"`
Gemini CLI	`--yolo`, or per server `"trust": true` in its `mcpServers` entry
n8n AI Agent	Don't add human-review steps to its tools

9.1 Give servers enough power

- **Remote OAuth servers** (GitHub, Notion, Linear…) act with *your* account's permissions. Approve every scope they request during sign-in.
- **Token-based servers:** create the token with the broad scopes from Chapter 8 (GitHub) or full capabilities (Notion, Chapter 11).
- **Local servers:** they run with your user's file and network permissions. Point filesystem servers at the folders you want writable.

Still asks: tools a server marks as needing user interaction (`requiresUserInteraction`) always prompt in Claude Code, and so do connector tools your organization set to "ask." ChatGPT's developer-mode connectors may still ask you to confirm write actions.

CHAPTER 10

n8n

10.1 Let AI agents build and run your workflows

OPEN N8N TO AI CLIENTS

1. **Create an API key:**
   `Settings` › `n8n API` › `Create an API key`. Copy it into scripts, Claude Code, or other tools that call n8n's REST API.

2. **Enable instance-level MCP:**
   `Settings` › `Instance-level MCP`, then **Enable MCP access** (owner or admin). Use **Connect** to get the server URL (ending in `/mcp-server/http`) and connect Claude, Cursor, Codex, or ChatGPT with OAuth or an API key.

3. **Expose workflows:**
   under **Workflows enabled**, click **Enable workflows** and add the ones agents may run, or in each workflow's **Settings** toggle **Available in MCP**. Clients can build and edit workflows (n8n 2.13+) as well as run them.

4. **Agents in workflows:**
   in AI Agent nodes, attach the tools you want and *don't* add human-in-the-loop approval to them.

10.2 Self-hosted: unlock everything

TERMINAL

```
# Re-enable nodes n8n 2.0+ blocks by default (Execute Command, etc.)
NODES_EXCLUDE="[]"
```

Set it in your Docker or Docker Compose environment and restart. **Execute Command** runs shell commands on the n8n host. It's powerful and dangerous, so keep self-hosted n8n in a container.

Still asks / limits: n8n Cloud doesn't offer Execute Command at all. The MCP server only lets clients run or see the full contents of workflows you enable for it (`search_workflows` previews everything you can access). Admins can disable MCP or Agents instance-wide.

CHAPTER 11

Notion

11.1 Notion MCP (simplest, full personal access)

Connect `https://mcp.notion.com/mcp` from Claude (Settings › Connectors › Notion), Claude Code, Cursor, or ChatGPT. Signing in with OAuth gives the AI exactly **your** access: every page you can see and edit. Then set its tools to **Always allow** in the host app (Chapter 9).

11.2 An API connection with every capability

For n8n, Zapier, and scripts:

CREATE AN ALL-ACCESS CONNECTION

1. **Open** notion.com/profile/integrations (or Settings › Connections › Develop or manage integrations) and create a new **internal connection** for your workspace. You must be a workspace owner.
2. **Enable every capability:**
   read content, update content, insert content, read comments, insert comments, and read user information including email addresses.
3. **Grant content access broadly.**
   In the connection's **Content access** tab, add every top-level page in each teamspace you want covered. Child pages inherit access.
4. **Copy the API token** into your tools.

11.3 Notion AI agents

Custom Agents run on schedules or triggers. Give each agent the sources and tools it needs in the Agent Builder, and review its run log rather than approving each action. On Business and Enterprise plans, admins control who can create agents, and Custom Agents use paid Notion credits.

Still limited: a connection can never see pages you haven't shared with it. Workspace owners can restrict or revoke MCP clients in `Settings` › `Connections`. Agents run inside Notion's own permission system and can't exceed the access of the content shared with them.

CHAPTER 12

What Always Asks: The Master List

These are the prompts and blocks that survive every setting in this guide. They're worth knowing so you're not surprised when a "fully autonomous" agent pauses.

TOOL	ALWAYS ASKS OR BLOCKS
Claude Code	Critical-path `rm` / `rmdir` (root, home, top-level dirs, working dir); explicit ask rules; `AskUserQuestion`; `requiresUserInteraction` MCP tools; org "ask" connector policies; PowerShell `Remove-Item` on system paths; running bypass as root outside a sandbox
Claude Desktop / Chrome	Purchases, account creation, CAPTCHAs; classifier-flagged actions on external sites; deleting sessions
Cowork	Write connector tools when the admin hasn't allowed "Always allow"
Claude cloud sessions	No bypass; Auto is the ceiling
Cursor	Team admin policies; hard-coded security rules; (outside Run Everything) protected paths and commands leaving the sandbox
VS Code / Copilot	Tools set to `false`; organization policies; enabled sandbox limits
Codex / Gemini CLI	Admin "secure mode" or disabled-YOLO settings; untrusted folders (Gemini)
GitHub	Sudo mode in the web UI; push protection for secrets; org SSO and policies
n8n	Workflows not enabled for MCP; Execute Command unavailable on Cloud
Notion	Pages not shared with a connection; owner-level MCP restrictions
ChatGPT agent	Confirmations before high-impact actions (purchases, sending); "watch mode" on sensitive sites

CHAPTER 13

Claude Code Permission Modes and Settings Reference

This chapter is the exhaustive reference behind Chapter 3. The tables were generated from Anthropic's permission-modes page (code.claude.com/docs/en/permission-modes) and settings reference (code.claude.com/docs/en/settings-reference) in October 2026.

IN THIS CHAPTER

- The six permission modes, compared
- Which mode to choose for each situation
- Every documented `settings.json` key, A–Z

13.1 The six permission modes

MODE	WHAT RUNS WITHOUT ASKING	BEST FOR
`default`	Reads only	Reviewing every action yourself, sensitive work
`acceptEdits`	Reads, file edits, and common filesystem commands (`mkdir`, `touch`, `mv`, `cp`, etc.)	Iterating on code you're reviewing
`plan`	Reads, plus classifier-approved commands when auto mode is available	Exploring a codebase before changing it
`auto`	Everything, with background safety checks	Long tasks, reducing prompt fatigue
`dontAsk`	File reads inside your working directories and pre-approved tools; anything that would prompt is denied	Locked-down CI and scripts
`bypassPermissions`	Everything	Isolated containers and VMs only

13.2 Choosing a mode

YOU WANT TO...	START WITH	ISOLATION NEEDED
Review every action yourself	`claude --permission-mode default`	None
Explore before changing anything	`claude --permission-mode plan`	None
Work hands-off with background safety checks	`claude --permission-mode auto`	None required. A sandbox or container adds defence in depth
Run in CI with an exact allowlist	`claude -p "run the test suite" --permission-mode dontAsk --allowedTools "Bash(npm test)" "Read"`	Your CI runner
Run fully unattended	`claude -p "<prompt>" --dangerously-skip-permissions`	**Required:** a container, VM, or sandbox runtime

Set a default for every session with `permissions.defaultMode` in `~/.claude/settings.json`, or in a project's `.claude/settings.json`. According to the docs, project settings can't make `auto` or `bypassPermissions` the starting mode, which stops a cloned repository from quietly granting itself autonomy.

> **DANGER**
>
> `bypassPermissions` means **every** action runs with no checks. Only use it inside something you're happy to throw away: a container, a VM, or a cloud sandbox with no access to your real accounts or files (Chapter 2).

13.3 Every settings key, A–Z

Search this table when a guide mentions a setting. Many keys only matter to administrators. Run `/config` or ask Claude (`/update-config`) to change settings safely.

KEY	WHAT IT DOES
`advisorModel`	Pick which model answers when Claude calls the server-side advisor tool.
`alwaysThinkingEnabled`	Turn extended thinking off for every session by setting this to `false`.
`availableModels`	Restrict which models people can select for the main session, subagents, skills, and the advisor.
`availableModelsMatch`	Choose how `availableModels` entries match model IDs.
`deniedModels`	Block specific models, with or without an `availableModels` allowlist and even when that list permits them.
`effortLevel`	Set a default effort level for models you haven't saved a level for.
`enforceAvailableModels`	The `/model` picker has a **Default** option, and `default` model setting describes the model it resolves to.
`fallbackModel`	Name backup models for Claude Code to try, in order, when your primary model is overloaded or unavailable.
`fastMode`	Turn fast mode on for sessions where it's available, for interactive work like rapid iteration or live debugging where you want speed at...
`fastModePerSessionOptIn`	Normally, running `/fast` saves `fastMode` to a person's user settings, so fast mode is on at the start of every later session.
`language`	Have Claude respond in a language other than English by default.
`maxEffortLevel`	Cap the effort level a session can use, leaving lower levels available.
`model`	Set the model every new session uses, so you don't have to pick one with `/model` each time.
`modelOverrides`	Map Anthropic model IDs to provider-specific model IDs, such as Amazon Bedrock inference profile ARNs.
`modelPicker`	List the models the `/model` picker offers, in the order you write them and under labels you choose, so the picker lists the models your...
`modelPricing`	Report spend at the rates your organization pays instead of list price.
`modelSettings`	Save an effort level for each model you use.
`outputStyle`	Select an output style by name.
`promptCacheTtl`	Choose how long the prompt cache holds the main conversation.
`showThinkingSummaries`	See summaries of Claude's extended thinking in interactive sessions.
`subagentPromptCacheTtl`	Choose how long the prompt cache holds the requests Claude Code makes outside the main conversation.
`switchModelsOnFlag`	Choose what happens when a safety classifier flags a request: switch to the fallback model and continue, or pause so you can choose...
`ultracode`	Start sessions with ultracode on.
`allowManagedPermissionRulesOnly`	Make managed settings the only settings source of permission rules.

`autoMode`	Add your own rules to what the auto mode classifier blocks and allows.
`autoMode.classifyAllShell`	Send every Bash and PowerShell command through the auto mode classifier while auto mode is active.
`disableAutoMode`	Remove auto mode from the `Shift+Tab` cycle.
`permissions`	Control which tools Claude can use without asking, which ones always prompt, and which ones are blocked, and set the permission mode a…
`useAutoModeDuringPlan`	Choose whether Claude Code uses the auto mode classifier to review shell commands in plan mode.
`permissions.allow`	List the tool uses Claude Code approves without asking you.
`permissions.ask`	List the tool uses that prompt you for confirmation even in a permission mode that would otherwise approve them, such as `acceptEdits`…
`permissions.deny`	List the tool uses Claude Code blocks.
`permissions.additionalDirectories`	Give Claude file access to directories outside the one you started in, as additional working directories.
`permissions.blockReadsOutsideWorkingDirectories`	Make Claude's file tools refuse reads outside your working directories in every permission mode, including `bypassPermissions`.
`permissions.defaultMode`	Set the permission mode new sessions start in.
`permissions.disableBypassPermissionsMode`	Prevent anyone from entering `bypassPermissions` mode.
`skipAutoPermissionPrompt`	Skip the one-time notice describing auto mode that Claude Code shows when you first enter auto mode yourself, for example through your…
`skipDangerousModePermissionPrompt`	Skip the confirmation dialog Claude Code shows before a session enters `bypassPermissions` mode, whether from…
`sandbox`	Isolate the Bash commands Claude runs from your filesystem and network with sandboxing.
`sandbox.enabled`	Turn on sandboxing for Bash commands.
`sandbox.failIfUnavailable`	Make Claude Code exit with an error at startup when `sandbox.enabled` is `true` but the sandbox can't start, because a dependency is…
`sandbox.autoAllowBashIfSandboxed`	Let Claude Code run sandboxed Bash commands without a permission prompt.
`sandbox.excludedCommands`	Name commands that Claude Code runs outside the sandbox, such as tools that don't work under it.
`sandbox.allowUnsandboxedCommands`	Let Claude retry a command outside the sandbox with the `dangerouslyDisableSandbox` parameter after the sandbox blocks it.
`sandbox.filesystem`	Control which paths sandboxed commands can read and write.
`sandbox.filesystem.allowWrite`	Add paths where sandboxed commands can write, beyond the working directory, the per-user temp directory, and the directories you've…

`sandbox.filesystem.denyWrite`	Block sandboxed commands from writing to specific paths, including paths inside a directory that is otherwise writable.
`sandbox.filesystem.denyRead`	Block sandboxed commands from reading specific paths, such as credential files that the default read policy would otherwise expose.
`sandbox.filesystem.allowRead`	Re-open reading for specific paths inside a region that `denyRead` blocks, to build workspace-only read access.
`sandbox.filesystem.allowManagedReadPathsOnly`	Honor only the `allowRead` entries that come from managed settings, so developers can't re-open read access to paths your organization...
`sandbox.filesystem.disabled`	Skip filesystem isolation while keeping network isolation.
`sandbox.ignoreViolations`	Silence sandbox violation reports for paths you expect a command to probe and be refused, such as a tool that checks `/etc/hosts` on...
`sandbox.enableWeakerNestedSandbox`	Run the Linux sandbox inside an unprivileged Docker container, where bubblewrap can't mount a fresh `/proc`.
`sandbox.enableWeakerNetworkIsolation`	Let sandboxed commands on macOS reach the system TLS trust service, `com.apple.trustd.agent`.
`sandbox.allowAppleEvents`	Let sandboxed commands on macOS send Apple Events, which `open`, `osascript`, and tools that open URLs in a browser need; without it...
`sandbox.ripgrep`	Point the sandbox at a ripgrep binary of your own instead of the one Claude Code uses, for example when your platform needs a...
`sandbox.bwrapPath`	Point the sandbox at a bubblewrap binary installed outside `PATH`, such as a vendored copy on an air-gapped host.
`sandbox.socatPath`	Point the sandbox network proxy at a `socat` binary installed outside `PATH`.
`sandbox.credentials`	Declare the credential files and environment variables to protect from sandboxed commands.
`sandbox.credentials.files`	Protect credential files or directories from sandboxed commands.
`sandbox.credentials.envVars`	Protect environment variables from sandboxed commands.
`sandbox.credentials.allowPlaintextInject`	Allow `mask` substitution on plain HTTP requests as well as TLS-terminated HTTPS. On plain HTTP the upstream identity is unverified and...
`sandbox.credentials.awsPairs`	Group masked environment variables that form one AWS credential for SigV4 re-signing when your credential lives in variables with...
`sandbox.credentials.sigv4`	Choose what the sandbox proxy does with AWS request forms it can't re-sign: `streaming` for aws-chunked streaming uploads, `presigned`...
`sandbox.network`	Control which hosts, ports, and sockets sandboxed commands can reach.
`sandbox.network.allowUnixSockets`	List the Unix socket paths sandboxed commands can connect to on macOS. Claude Code ignores this list on Linux and WSL2, where the...
`sandbox.network.allowAllUnixSockets`	Let sandboxed commands connect to every Unix socket.
`sandbox.network.allowLocalBinding`	Let sandboxed commands on macOS listen on network ports, for example to start a dev server, and connect to any port on localhost.
`sandbox.network.allowMachLookup`	List additional XPC and Mach service names the macOS sandbox may look up.

`sandbox.network.allowedDomains`	Pre-allow domains for outbound traffic from sandboxed commands, so the sandbox doesn't prompt for them.
`sandbox.network.deniedDomains`	Block domains for outbound traffic from sandboxed commands, using the same wildcard, port, and IPv6 syntax as `allowedDomains`.
`sandbox.network.strictAllowlist`	Deny sandboxed commands access to hosts outside the allowlist instead of prompting for approval.
`sandbox.network.allowManagedDomainsOnly`	Lock the network allowlist to what managed settings define.
`sandbox.network.httpProxyPort`	Point the sandbox at your own HTTP proxy instead of the one Claude Code runs.
`sandbox.network.socksProxyPort`	Point the sandbox at your own SOCKS5 proxy instead of the one Claude Code runs.
`sandbox.network.tlsTerminate`	Make the sandbox proxy terminate TLS so it can read the contents of HTTPS requests.
`autoCompactEnabled`	Have Claude Code compact the conversation automatically when context approaches the limit.
`autoCompactWindow`	Set how full the context window gets before Claude Code compacts automatically.
`autoMemoryDirectory`	Store auto memory in a directory of your choice instead of the per-project default.
`autoMemoryEnabled`	Turn auto memory on or off.
`bashOutputMaxChars`	Set how many characters of a successful Bash or PowerShell command's output Claude receives inline.
`claudeMd`	Inject CLAUDE.md-style instructions as organization-managed memory without deploying a separate file.
`claudeMdExcludes`	Skip specific `CLAUDE.md` files when Claude Code loads memory.
`env`	Set environment variables for every session and for the subprocesses Claude Code starts from it.
`fileCheckpointingEnabled`	Have Claude Code snapshot files before each edit so `/rewind` can restore them.
`idleCompaction`	Idle compaction compacts a long conversation that you've stepped away from, shortly before its prompt cache expires.
`plansDirectory`	Choose where Claude Code stores the plan files it writes in plan mode.
`skillListingBudgetFraction`	Each turn, Claude sees a listing of your skills with their descriptions, and Claude Code caps that listing at a share of the context window.
`skillListingMaxDescChars`	Each turn, Claude sees a listing of your skills that shows each skill's `description` and `when_to_use` text.
`taskOutputMaxChars`	Through v2.1.276, you set this key to the number of characters of a background task's output that Claude received inline when it read...
`askUserQuestionTimeout`	Let an unanswered `AskUserQuestion` dialog auto-continue after a period of idle time, submitting whatever options you had already selected.
`autoContinueAtUsageLimit`	After a claude.ai usage limit stops your session, wait in the open session and continue the task automatically after the reset.
`autoScrollEnabled`	Follow new output to the bottom of the conversation in fullscreen rendering.

`axScreenReader`	Render screen-reader friendly output: flat text without decorative borders or animations.
`bashEditDiffEnabled`	Choose whether Claude Code records which files changed in a Git repository while a Bash command runs.
`companyAnnouncements`	Show your organization's announcements to users at startup.
`defaultShell`	Choose whether Bash or PowerShell runs the shell commands you type with the `!` prefix in the input box, the ones Claude Code runs...
`dialogExpiry`	Set the deadline for dialogs Claude Code forwards to a remote client, such as a Remote Control or SDK host, and for the approval dialog...
`editorMode`	Choose the key binding mode for the input prompt.
`emojiCompletionEnabled`	Show emoji suggestions when you type `:` plus a shortcode in the prompt input, and replace a completed shortcode such as `:heart:` with...
`fileSuggestion`	Run your own command to supply `@` file path autocomplete instead of the built-in file suggestion.
`footerLinksRegexes`	Render extra clickable badges in the footer below the input box when a regex matches turn output: tool results, including file contents...
`keybindingFlavor`	In v2.1.238 through v2.1.260, setting it to `"readline"` made `Ctrl+W` delete back to the previous whitespace instead of only the...
`maxProseWidth`	Cap the width of the prose in Claude's responses so lines stay readable in a wide terminal.
`prefersReducedMotion`	Reduce or turn off interface animations such as the spinner, shimmer, and flash effects.
`promptSuggestionEnabled`	Show or hide prompt suggestions, the grayed-out predictions that appear in your prompt input.
`respectGitignore`	Control whether the `@` file picker leaves out files that match `.gitignore` patterns.
`respondToBashCommands`	Choose whether Claude responds after you run a shell command with the `!` prefix in the input box.
`showClearContextOnPlanAccept`	When Claude finishes a plan in plan mode, it shows an approval menu.
`showTurnDuration`	Show or hide the turn duration message after each response, such as "Cooked for 1m 6s · done 6:05 PM". The clock after "done" shows when...
`spellcheck`	Underline misspelled words in the prompt input as you type, using a spell checker you install.
`spinnerTipsEnabled`	While Claude works, the spinner line rotates through short tips about Claude Code features, such as "Use Plan Mode to prepare for a...
`spinnerTipsOverride`	Add your own tips to the spinner tips that Claude Code shows while Claude works, or replace the built-in tips with yours.
`spinnerVerbs`	While a turn is in progress, the spinner shows a rotating verb such as "Accomplishing", "Architecting", or "Baking". Use this key to add...
`statusLine`	Run your own command to render a status line below the prompt with context such as the model, cost, or git branch.
`subagentStatusLine`	When Claude runs subagents, Claude Code lists them in a task display below the prompt, one row per subagent showing `name · description...
`syntaxHighlightingDisabled`	Claude Code colors code by language in the diffs, code blocks, and file previews it shows in the terminal, with its built-in...
`terminalProgressBarEnabled`	Some terminals can show a progress indicator on the tab or in the taskbar for the program running in them.

`terminalTitleFromRename`	Claude Code sets your terminal tab's title.
`theme`	Pick the color theme for the interface.
`timeFormat`	Choose how Claude Code writes the times it shows in the interface, such as the `done 6:05 PM` at the end of each turn duration message…
`timeZone`	Show the times in the interface in a time zone other than your system's.
`tui`	Choose the terminal UI renderer.
`verbose`	By default, the transcript collapses each tool call to a short summary, such as the command Claude ran and a line count of its output,…
`viewMode`	Set the transcript view Claude Code starts in: `"default"`, `"verbose"`, or `"focus"`.
`vimInsertModeRemaps`	Map two-key INSERT-mode sequences to Escape in vim editor mode.
`voice`	Turn on voice dictation and choose how the dictation key behaves.
`voiceEnabled`	Turn voice dictation on with the single Boolean form that predates the `voice` object.
`wheelScrollAccelerationEnabled`	Accelerate mouse-wheel scroll speed during fast scrolls in fullscreen rendering.
`attribution`	Customize the attribution Claude Code adds to git commits and pull requests.
`includeCoAuthoredBy`	Use `attribution` instead, which replaces this key and lets you change or hide the commit trailer, the pull request text, and the…
`includeGitInstructions`	Claude Code gives Claude two git-related pieces of context: its built-in instructions for how to write commits and pull requests, in the…
`prUrlTemplate`	Point the PR links Claude Code renders, in the footer badge and in tool-result summaries, at an internal code-review tool instead of…
`attribution.commit`	Set the attribution text Claude Code adds to git commits, including any trailers.
`attribution.pr`	Set the attribution text Claude Code adds to pull request descriptions.
`attribution.sessionUrl`	Choose whether Claude Code appends the claude.ai session link when it commits or opens a pull request from a cloud or Remote Control…
`allowedHttpHookUrls`	Limit which URLs HTTP hooks can target.
`allowManagedHooksOnly`	Restrict hook execution to hooks your organization deploys.
`disableAllHooks`	Turn off hooks, any custom status line, and any custom file suggestion command.
`disableWorkflows`	Turn off dynamic workflows and the bundled workflow commands for everyone your settings reach, such as an organization through managed…
`enableWorkflows`	Turn dynamic workflows on or off for yourself when your plan's default isn't what you want.
`hooks`	Run your own commands, prompts, agents, HTTP requests, or MCP tools as hooks at points in Claude Code's lifecycle, such as before a tool…
`httpHookAllowedEnvVars`	An HTTP hook can put the value of an environment variable into a request header, for example an `Authorization: Bearer $HOOK_TOKEN`…
`workflowKeywordTriggerEnabled`	Choose whether typing the keyword `ultracode` in a prompt triggers a dynamic workflow.
`workflowSizeGuideline`	Set the agent count Claude aims for in the dynamic workflows it writes.
`disableBundledSkills`	Turn off the skills and workflows included with Claude Code.
`disableSkillShellExecution`	Turn off inline shell execution for ``!`...` `` and ``` ```! ``` blocks in skills and custom commands from user, project, plugin, or…

`skillOverrides`	Hide or collapse a skill without editing its `SKILL.md`.
`syncClaudeAiSkills`	Turn off the download of the skills enabled for your claude.ai account.
`syncClaudeAiPlugins`	Turn off the download of the plugins enabled for your claude.ai account.
`allowedChannelPlugins`	Choose which channel plugins can push messages into sessions in your organization.
`blockedMarketplaces`	Block plugin marketplace sources for your organization.
`channelsEnabled`	Allow channels for your organization.
`disableCommandPluginSources`	Block the `command` plugin source, which installs a plugin by running a marketplace-declared command on the user's machine.
`pluginSuggestionMarketplaces`	Name the marketplaces whose plugins can appear as contextual install suggestions, in spinner tips and pinned at the top of the `/plugin` ...
`pluginTrustMessage`	Add your organization's own text to the plugin trust warning Claude Code shows before installation, for example to confirm that plugins...
`strictKnownMarketplaces`	Restrict which plugin marketplace sources people in your organization can add and install plugins from.
`strictPluginOnlyCustomization`	Block skills, agents, hooks, and MCP servers from user and project sources, so they can come only from plugins or managed settings.
`strictPluginOnlyCustomization.skills`	Lock the `skills` surface.
`strictPluginOnlyCustomization.agents`	Lock the `agents` surface.
`strictPluginOnlyCustomization.hooks`	Lock the `hooks` surface.
`strictPluginOnlyCustomization.mcp`	Lock the `mcp` surface.
`enabledPlugins`	Turn individual plugins on or off, keyed by `plugin-name@marketplace-name`.
`extraKnownMarketplaces`	Register additional plugin marketplaces by name, so that people who open the repository, or everyone your managed settings reach, get...
`pluginConfigs`	Store the non-sensitive answers you give a plugin's `userConfig` configuration dialog, keyed by plugin ID. Claude Code writes this key...
`prependPlugins`	List the managed plugins whose mods run before every mod a user installs, in the listed order.
`appendPlugins`	List the managed plugins whose mods run after every mod a user installs, in the listed order.
`allowAllClaudeAiMcps`	Load the claude.ai connectors Claude Code fetches itself alongside a deployed `managed-mcp.json`.
`allowClaudeInChromeWithManagedMcp`	Let the built-in Claude in Chrome server run alongside a deployed `managed-mcp.json`.
`allowedMcpServers`	Allowlist the MCP servers people can add.
`allowManagedMcpServersOnly`	Make the managed allowlist the only one that applies.
`deniedMcpServers`	Block specific MCP servers.
`disableClaudeAiConnectors`	Turn off the claude.ai MCP connectors Claude Code fetches itself, so it neither fetches nor connects them.
`disabledMcpjsonServers`	Reject specific servers defined in a project's `.mcp.json` file so Claude Code never connects them or asks you to approve them.

`enableAllProjectMcpServers`	Approve every MCP server defined in project `.mcp.json` files without a prompt.
`enabledMcpjsonServers`	Approve specific servers defined in project `.mcp.json` files so Claude Code connects them without asking.
`managedMcpServers`	Provide remote MCP servers to every user from managed settings.
`agent`	Run the main thread as a named subagent, so Claude Code applies that subagent's system prompt, tool restrictions, and model to your session.
`crossSessionInbound`	Choose what this session does with messages arriving from your other Claude Code sessions.
`disableAgentView`	Turn off background agents and agent view: `claude agents`, `--bg`, `/background`, and the on-demand supervisor.
`isolatePeerMachines`	Require your explicit approval before Claude's `SendMessage` reaches one of your sessions beyond this machine; see Require approval for...
`processWrapper`	On macOS and Linux, place a corporate launcher command in front of the background processes Claude Code starts.
`teammateMode`	Choose where Claude Code shows agent team teammates: inside your main terminal pane, or in split panes when your terminal supports them.
`worktree`	Configure how Claude Code creates and manages git worktrees for `--worktree`, the `EnterWorktree` tool, and isolated subagents and...
`worktree.baseRef`	Choose which ref new worktrees branch from. `"fresh"` branches from `origin/` for a clean tree matching the remote; `"head"` branches...
`worktree.symlinkDirectories`	Symlink directories from the main repository into each worktree so you don't duplicate large directories on disk.
`worktree.sparsePaths`	Check out only the listed directories in each worktree through git sparse-checkout.
`worktree.bgIsolation`	Choose how background sessions isolate their file edits.
`worktree.location`	Choose the folder on a remote machine where Desktop SSH sessions create their worktrees, instead of `/.claude/worktrees/`.
`agentPushNotifEnabled`	Allow Claude to send a push notification to your phone when it decides one is worth sending, for example when a long task finishes.
`awaySummaryEnabled`	Show a one-line session recap when you return to the terminal after a few minutes away.
`disableArtifact`	Use `enableArtifact` instead to turn off the Artifact tool, which publishes session output as a private web page on claude.ai.
`disableDeepLinkRegistration`	Stop Claude Code from registering the `claude-cli://` protocol handler with the operating system, which it otherwise does after you send...
`disableDesktopLocalSessions`	Turn off Code sessions that run on the device in the desktop app, for deployments where developers should work on remote machines over...
`disableRemoteControl`	Turn off Remote Control: Claude Code then refuses `claude remote-control`, the `--remote-control` flag, auto-start, and the in-session...
`enableArtifact`	Turn off the Artifact tool, which publishes session output as a private web page on claude.ai.
`inputNeededNotifEnabled`	Get a push notification on your phone when a permission prompt or question is waiting for your input.
`preferredNotifChannel`	Choose how Claude Code notifies you when a task completes or a permission prompt is waiting.
`remote.defaultEnvironmentId`	Pick the default cloud environment for cloud sessions you create from the CLI, such as with `claude --cloud`.

`remoteControlAtStartup`	Connect Remote Control automatically when each interactive session starts, instead of waiting for `/remote-control`.
`sshConfigs`	Add SSH connections to the Desktop environment dropdown.
`sshHostAllowlist`	Limit the hosts a Desktop SSH session can connect to.
`allowedProviders`	List the services a machine may reach Claude through, such as the Anthropic API, Amazon Bedrock, or an LLM gateway.
`apiKeyHelper`	Run your own command to produce the credential Claude Code sends with model requests.
`awsAuthRefresh`	Run your own command, such as `aws sso login`, to refresh the credentials in your `.aws` directory when the ones Claude Code has for…
`awsCredentialExport`	Run your own command that prints AWS credentials as JSON, so Claude Code can call Amazon Bedrock with credentials that don’t live in…
`forceLoginMethod`	Restrict which kind of account people can log in with.
`forceLoginGatewayUrl`	Set the gateway URL the `/login` Cloud gateway screen connects to, so people reach your cloud gateway without typing its address.
`forceLoginOrgUUID`	From a managed source, require claude.ai account logins to belong to one Anthropic organization, given as a single UUID, or to any of…
`gatewayInternalNetworks`	Declare the public IPv4 blocks that your organization numbers its internal network from, so `/login` accepts a cloud gateway there.
`gcpAuthRefresh`	Run your own command to refresh Google Cloud Application Default Credentials when Claude Code finds they’ve expired or can’t be loaded,…
`otelHeadersHelper`	Run your own command to generate the headers Claude Code sends with OpenTelemetry exports, for backends whose tokens rotate.
`autoUpdatesChannel`	Choose which release channel background auto-updates and `claude update` follow.
`minimumVersion`	Keep background auto-updates and `claude update` from installing any version below this one, so moving to the `"stable"` channel doesn’t…
`requiredMaximumVersion`	Set the newest Claude Code version your organization allows to start.
`requiredMinimumVersion`	Set the oldest Claude Code version your organization allows to start.
`browserExternalPageTools`	Stop Claude from using its tools to read or act on external pages in the desktop app’s Browser pane.
`disableBrowserExternalNavigation`	Turn off external browsing in the desktop app’s Browser pane for people and Claude alike.
`disableMobileSimulatorTools`	Block Claude’s tools for the desktop app’s iOS Simulator pane.
`cleanupPeriodDays`	Set how many days Claude Code keeps session transcripts and other application data before deleting them.
`desktopSessionCleanupPeriodDays`	Set an age limit in days for the transcripts of sessions you started or most recently continued in Claude Desktop or Cowork.
`feedbackDrafts`	Control Claude-drafted feedback: whether Claude can queue feedback drafts for you to review, and whether Claude Code shows a card when…
`feedbackSurveyRate`	Set the probability that the session quality survey appears when a session is eligible for it.
`skipWebFetchPreflight`	Skip the WebFetch domain safety check, which sends each requested hostname to `api.anthropic.com` before fetching.

`disableSideloadFlags`	Reject the `--plugin-dir`, `--plugin-url`, `--agents`, and `--mcp-config` CLI flags at startup, which users could otherwise pass to...
`forceRemoteSettingsRefresh`	Block CLI startup until Claude Code has freshly fetched server-managed settings.
`managedSourcesBehavior`	Choose whether Claude Code applies only the highest-priority managed source your organization delivers, or combines every admin source...
`parentSettingsBehavior`	Choose whether Claude Code applies managed settings supplied by an embedding host process, such as the Agent SDK or an IDE extension,...
`policyHelper`	Run an executable you deploy that computes managed settings at startup, so you can derive policy from device posture, identity, or a...
`policyHelper.path`	Name the helper executable Claude Code runs.
`policyHelper.timeoutMs`	Set how long Claude Code waits for the helper before treating the run as failed.
`policyHelper.refreshIntervalMs`	Have Claude Code re-run the helper in the background on an interval so policy changes reach a running session.
`wslInheritsWindowsSettings`	Have Claude Code on WSL read managed settings from the Windows policy chain, with HKLM and the Windows managed settings file taking...
`autoConnectIde`	Connect to a running IDE automatically when you start Claude Code from an external terminal.
`autoInstallIdeExtension`	Install the Claude Code IDE extension automatically when you run Claude Code from a VS Code terminal.
`claudeInChromeDefaultEnabled`	Start every interactive CLI session with Chrome integration on, without passing `--chrome` each time.
`copyFullResponse`	Make `/copy` copy the full response every time, without showing the picker.
`copyOnSelect`	Copy text to your clipboard automatically when you finish selecting it with the mouse in fullscreen rendering or agent view.
`defaultToAgentsView`	Open agent view instead of a new conversation when you run `claude` with no arguments.
`diffTool`	Choose where Claude Code shows the diff of an `Edit` or `Write` change it proposes when a VS Code or JetBrains IDE is connected:...
`externalEditorContext`	When you press `Ctrl+G`, Claude Code opens the prompt you're typing in your external editor.
`leftArrowOpensAgents`	Press `←` on an empty prompt to background the session and open agent view.
`permissionExplainerEnabled`	Through v2.1.256, you could press `Ctrl+E` on a Bash or PowerShell permission prompt to see a model-generated explanation of the...
`prStatusFooterEnabled`	Show a badge in the prompt footer for the current branch's open pull request or merge request, with a colored underline that shows its...
`teammateDefaultModel`	Through v2.1.233, you set this key to the model for agent team teammates your prompt didn't name a model for: an alias such as...

CHAPTER 14

Autonomy Recipes

Copy-ready setups for the most common hands-off scenarios. Each recipe lists the risk level and the guardrails you must keep.

RECIPE	SETUP	RISK	MUST-KEEP GUARDRAILS
Overnight refactor in a container	Dev container or VM, `--dangerously-skip-permissions`, a git branch	Medium	No production credentials in the box. Review the diff in the morning
Daily docs or report job	`claude -p` from cron, `dontAsk`, an allowlist of exact commands	Low	Read-only tokens. Output to a file, not straight to publishing
PR auto-fix bot	CI runner, allowlist of test and lint commands, a scoped GitHub token	Medium	Branch protection on `main`. A human merges
n8n AI agent with tools	Agent node with minimal tools, approval step for sending or deleting	Medium	Error workflow, rate limits, logging
Notion assistant	Integration shared with one teamspace only	Low	No delete permission. Weekly audit
Browser agent	Separate browser profile, no saved passwords	High	Watch it live. Never let it handle payments unattended
Trading bot	Paper account only (Vol. 10)	High	Kill switch, loss limits, separate keys

14.1 Golden rules of autonomy

- [ ] Autonomy goes in a box: a container, VM, or separate account
- [ ] Credentials are scoped to the task and set to expire
- [ ] Anything that can’t be undone gets a human approval step
- [ ] Everything is logged, and logs are reviewed
- [ ] There’s a tested kill switch for every agent
- [ ] Start with a short leash and lengthen it only after a clean track record

CHAPTER 15

Incident Response: When an Agent Goes Wrong

Sooner or later an agent will do something you didn't expect. A calm, practised routine turns a disaster into an inconvenience.

STOP, CONTAIN, RECOVER

1. **Stop it.**
   Press `Esc` or `Ctrl` + `C`, stop the container or VM, deactivate the n8n workflow, or use `/stop` for a background session.

2. **Cut its access.**
   Revoke or rotate any tokens it held: GitHub, cloud, Notion, MCP connectors, API keys.

3. **Assess.**
   Check `git status` and `git log`, cloud consoles, sent mail, and billing pages. What changed?

4. **Restore.**
   `git revert` or restore from a branch or backup. Use `/rewind` for local file edits. Restore databases from snapshots.

5. **Notify.**
   If other people's data or money was affected, tell them, and the relevant service.

6. **Learn.**
   Write down what permission made it possible, then tighten that rule or add a hook or deny rule.

Emergency card: Esc / Ctrl+C → kill container → revoke tokens → check git, billing, and sent mail → revert → tighten rules.

CHAPTER 16

Codex config.toml Reference

This chapter lists every key in Codex's `~/.codex/config.toml` and the admin-only `requirements.toml`. It was generated on October 16, 2026 from the official reference at developers.openai.com/codex/config-reference, and descriptions are shortened to their first sentence. Check the official page for full details before you change a key you don't recognize.

- **User config:** `~/.codex/config.toml`. **Project config:** `.codex/config.toml`, which loads only when you trust the project.
- Project config can't override provider, auth, notification, profile or telemetry keys. Codex ignores them there.
- `approval_policy = "untrusted"` is **retired**. Use `on-request`, `never`, or a `granular` table.
- **Profiles** live next to config.toml as `$CODEX_HOME/<name>.config.toml`. Select one with `--profile <name>`.

16.1 Command-line autonomy flags (current release)

These come from the Codex source (`codex-rs/utils/cli/src/shared_options.rs`).

FLAG	ALIAS	WHAT IT DOES
`--sandbox <mode>`, `-s`	—	Sets the sandbox: `read-only`, `workspace-write` or `danger-full-access`
`--approve-for-me`	`--not-so-yolo`	Same as `approvals_reviewer="auto_review"`, `approval_policy="on-request"` and `sandbox_mode="workspace-write"`
`--dangerously-bypass-approvals-and-sandbox`	`--yolo`	No prompts and no sandbox. Use it only inside a sandbox you built yourself
`--dangerously-bypass-hook-trust`	—	Runs enabled hooks without persisted hook trust for this one run
`--profile <name>`, `-p`	—	Layers `$CODEX_HOME/<name>.config.toml` on top of your config
`--cd <dir>`, `-C`	—	Sets the working root
`--add-dir <dir>`	—	Adds another writable directory
`--worktree`	—	Runs the session in a new managed Git worktree

> ⚠ **WARNING**
> `--full-auto` appears in older tutorials but no longer exists. Use `--sandbox workspace-write` or `--approve-for-me` instead.

16.2 config.toml keys (292)

KEY	TYPE	WHAT IT DOES
`model`	`string`	Model to use (e.g., `gpt-6.1-sol`).
`review_model`	`string`	Optional model override used by `/review` (defaults to the current session model).
`model_provider`	`string`	Provider id from `model_providers` (default: `openai`).
`openai_base_url`	`string`	Base URL override for the built-in `openai` model provider.
`model_context_window`	`number`	Context window tokens available to the active model.
`model_auto_compact_token_limit`	`number`	Token threshold that triggers automatic history compaction (unset uses model defaults).
`model_auto_compact_token_limit_scope`	`total / body_after_prefix`	Controls whether the auto-compaction threshold counts the full active context (`total`, the default) or only growth after the…
`model_catalog_json`	`string (path)`	Optional path to a JSON model catalog loaded on startup.
`oss_provider`	`lmstudio / ollama`	Default local provider used when running with `--oss` (defaults to prompting if unset).

`approval_policy`	`on-request / never / { granular = { s…`	Controls when Codex pauses for approval before executing commands.
`approval_policy.granular.sandbox_approval`	`boolean`	When `true`, sandbox escalation approval prompts are allowed to surface.
`approval_policy.granular.rules`	`boolean`	When `true`, approvals triggered by execpolicy `prompt` rules are allowed to surface.
`approval_policy.granular.mcp_elicitations`	`boolean`	When `true`, MCP elicitation prompts are allowed to surface instead of being auto-rejected.
`approval_policy.granular.request_permissions`	`boolean`	When `true`, prompts from the `request_permissions` tool are allowed to surface.
`approval_policy.granular.skill_approval`	`boolean`	When `true`, skill-script approval prompts are allowed to surface.
`approvals_reviewer`	`user / auto_review`	Who reviews eligible approval prompts under `on-request` or granular approval policies.
`auto_review.policy`	`string`	Local Markdown policy instructions for automatic review.
`auto_review.extra_policy`	`string`	Additional local Markdown policy for automatic review, included alongside the main policy.
`allow_login_shell`	`boolean`	Allow shell-based tools to use login-shell semantics.
`sandbox_mode`	`read-only / workspace-write / danger-…`	Sandbox policy for filesystem and network access during command execution.
`sandbox_workspace_write.network_access`	`boolean`	Allow outbound network access inside the workspace-write sandbox.
`sandbox_workspace_write.exclude_tmpdir_env_var`	`boolean`	Exclude `$TMPDIR` from writable roots in workspace-write mode.
`sandbox_workspace_write.exclude_slash_tmp`	`boolean`	Exclude `/tmp` from writable roots in workspace-write mode.
`windows.sandbox`	`mxc / elevated / unelevated`	Native Windows sandbox implementation.
`browser_use.allow_history_access`	`boolean`	Set to `false` to restrict browser-history access.
`browser_use.default_origin_policy`	`table`	Fallback browser-origin restrictions.
`browser_use.origins.<origin>`	`table`	Per-origin browser restrictions with the same fields as `browser_use.default_origin_policy`.
`computer_use.default_app_access`	`allow / deny`	Fallback native-app access policy for Computer Use.
`computer_use.macos.bundle_ids`	`map<string, allow / deny>`	Native macOS app access keyed by bundle identifier.

`computer_use.windows.aumids`	`map<string, allow / deny>`	Packaged Windows app access keyed by Application User Model ID (AUMID).
`computer_use.windows.exes`	`array<table>`	Windows executable access rules.
`computer_use.windows.always_allowed_app_ids`	`array<string>`	Windows app identifiers that Computer Use can open without prompting.
`notify`	`array<string>`	Command invoked for notifications; receives a JSON payload from Codex.
`check_for_update_on_startup`	`boolean`	Check for Codex updates on startup (set to false only when updates are centrally managed).
`feedback.enabled`	`boolean`	Enable feedback submission via `/feedback` across local clients (default: true).
`analytics.enabled`	`boolean`	Enable or disable analytics for this machine/profile.
`instructions`	`string`	Reserved for future use; prefer `model_instructions_file` or `AGENTS.md`.
`developer_instructions`	`string`	Additional developer instructions injected into the session (optional).
`log_dir`	`string (path)`	Directory where Codex writes log files; defaults to `$CODEX_HOME/log`.
`sqlite_home`	`string (path)`	Directory where Codex stores the SQLite-backed state DB used by agent jobs and other resumable runtime state.
`compact_prompt`	`string`	Inline override for the history compaction prompt.
`model_instructions_file`	`string (path)`	Replacement for built-in instructions instead of `AGENTS.md`.
`personality`	`none / friendly / pragmatic`	Deprecated compatibility setting. `friendly` and `pragmatic` no longer select a style. `none` omits the built-in Personality...
`service_tier`	`string`	Preferred service tier for new turns.
`experimental_compact_prompt_file`	`string (path)`	Load the compaction prompt override from a file (experimental).
`skills.max_context_tokens`	`integer (positive)`	Token budget for the available-skills catalog.
`skills.config`	`array<object>`	Per-skill enablement overrides stored in config.toml.
`skills.config.<index>.path`	`string (path)`	Path to a skill folder containing `SKILL.md`.
`skills.config.<index>.enabled`	`boolean`	Enable or disable the referenced skill.
`apps.<id>.enabled`	`boolean`	Enable or disable a specific app/connector by id (default: true).
`apps._default.enabled`	`boolean`	Default app enabled state for all apps unless overridden per app.
`apps._default.destructive_enabled`	`boolean`	Default allow/deny for app tools with `destructive_hint = true`.

`apps._default.open_world_enabled`	`boolean`	Default allow/deny for app tools with `open_world_hint = true`.
`apps._default.approvals_reviewer`	`user / auto_review`	Default reviewer for app tool approval prompts unless overridden per app.
`apps._default.default_tools_approval_mode`	`auto / prompt / writes / approve`	Default approval behavior for app tools without per-app or per-tool overrides.
`apps.<id>.destructive_enabled`	`boolean`	Allow or block tools in this app that advertise `destructive_hint = true`.
`apps.<id>.open_world_enabled`	`boolean`	Allow or block tools in this app that advertise `open_world_hint = true`.
`apps.<id>.default_tools_enabled`	`boolean`	Default enabled state for tools in this app unless a per-tool override exists.
`apps.<id>.approvals_reviewer`	`user / auto_review`	Reviewer for this app's tool approval prompts.
`apps.<id>.default_tools_approval_mode`	`auto / prompt / writes / approve`	Default approval behavior for tools in this app unless a per-tool override exists.
`apps.<id>.tools.<tool>.enabled`	`boolean`	Per-tool enabled override for an app tool (for example `repos/list`).
`apps.<id>.tools.<tool>.approval_mode`	`auto / prompt / writes / approve`	Per-tool approval behavior override for a single app tool.
`features.apps`	`boolean`	Enable app (connector) integrations (stable; on by default).
`features.hooks`	`boolean`	Enable lifecycle hooks loaded from `hooks.json` or inline `[hooks]` config. `features.codex_hooks` is a deprecated alias.
`features.code_mode.enabled`	`boolean`	Enable code mode feature configuration.
`features.code_mode.excluded_tool_namespaces`	`array<string>`	Tool namespaces code mode excludes from nested code-mode tool guidance and executor exposure.
`features.code_mode.direct_only_tool_namespaces`	`array<string>`	Tool namespaces code mode can use only through direct tool calls.
`features.context_management.experimental_mode`	`boolean`	Experimental context-management setting.
`features.rollout_budget.enabled`	`boolean`	Enable rollout budget tracking.
`features.rollout_budget.limit_tokens`	`integer`	Positive token limit for rollout budget tracking.
`features.rollout_budget.reminder_interval_tokens`	`integer`	Positive token interval between rollout budget reminders.
`features.rollout_budget.sampling_token_weight`	`number`	Finite non-negative multiplier for sampled tokens in rollout budget accounting.
`features.rollout_budget.prefill_token_weight`	`number`	Finite non-negative multiplier for prefill tokens in rollout budget accounting.
`hooks`	`table`	Lifecycle hooks configured inline in `config.toml`.

`hooks.<Event>`	`array<table>`	Matcher groups for hook events such as `PreToolUse`, `PermissionRequest`, `PostToolUse`, `PreCompact`, `PostCompact`,...
`hooks.<Event>[].hooks`	`array<table>`	Hook handlers for a matcher group.
`hooks.<Event>[].hooks[].async`	`boolean`	Run a command hook in the background without delaying the triggering operation.
`hooks.<Event>[].hooks[].additionalContextLimit`	`integer`	Approximate per-handler token threshold for saving oversized `additionalContext` to disk and showing the model a shorter preview.
`hooks.<Event>[].hooks[].commandWindows`	`string`	Windows-only command override for command hooks.
`features.memories`	`boolean`	Enable Memories (off by default).
`mcp_optional_startup_grace_ms`	`integer (milliseconds)`	Shared wait for optional MCP servers when building the initial tool catalog.
`mcp_servers.<id>.command`	`string`	Launcher command for an MCP stdio server.
`mcp_servers.<id>.args`	`array<string>`	Arguments passed to the MCP stdio server command.
`mcp_servers.<id>.env`	`map<string,string>`	Environment variables forwarded to the MCP stdio server.
`mcp_servers.<id>.cwd`	`string`	Working directory for the MCP stdio server process.
`mcp_servers.<id>.url`	`string`	Endpoint for an MCP streamable HTTP server.
`mcp_servers.<id>.auth`	`oauth / chatgpt`	Authentication fallback for an MCP HTTP server after configured bearer tokens and authorization headers. `oauth` (default)...
`mcp_servers.<id>.oauth.client_id`	`string`	Pre-registered OAuth client ID used for authorization and token exchange with this MCP server.
`mcp_servers.<id>.oauth.client_secret`	`string`	OAuth client secret for token exchange with a pre-registered client.
`mcp_servers.<id>.oauth.callback_url`	`string`	Server-specific OAuth callback.
`mcp_servers.<id>.oauth.callback_port`	`integer`	Fixed OAuth callback listener port for this MCP server.
`mcp_servers.<id>.bearer_token_env_var`	`string`	Environment variable sourcing the bearer token for an MCP HTTP server.
`mcp_servers.<id>.http_headers`	`map<string,string>`	Static HTTP headers included with each MCP HTTP request.
`mcp_servers.<id>.http_headers_helper`	`string (command)`	Local command that prints a JSON object of HTTP header names and values.
`mcp_servers.<id>.env_http_headers`	`map<string,string>`	HTTP headers populated from environment variables for an MCP HTTP server.
`mcp_servers.<id>.enabled`	`boolean`	Disable an MCP server without removing its configuration.
`mcp_servers.<id>.required`	`boolean`	When true, fail startup/resume if this enabled MCP server cannot initialize.

`mcp_servers.<id>.startup_timeout_sec`	`number`	Override the default 10s startup timeout for an MCP server.
`mcp_servers.<id>.startup_timeout_ms`	`number`	Alias for `startup_timeout_sec` in milliseconds.
`mcp_servers.<id>.tool_timeout_sec`	`number`	Override the default 60s per-tool timeout for an MCP server.
`mcp_servers.<id>.enabled_tools`	`array<string>`	Allow list of tool names exposed by the MCP server.
`mcp_servers.<id>.disabled_tools`	`array<string>`	Deny list applied after `enabled_tools` for the MCP server.
`mcp_servers.<id>.default_tools_approval_mode`	`auto / prompt / writes / approve`	Default approval behavior for MCP tools on this server unless a per-tool override exists.
`mcp_servers.<id>.tools.<tool>.approval_mode`	`auto / prompt / writes / approve`	Per-tool approval behavior override for one MCP tool on this server.
`mcp_servers.<id>.tools.<tool>.output_token_limit`	`integer (positive)`	Token budget for one MCP tool's output, before the standard 20% serialization allowance.
`mcp_servers.<id>.scopes`	`array<string>`	OAuth scopes to request when authenticating to that MCP server.
`mcp_servers.<id>.oauth_resource`	`string`	Optional RFC 8707 OAuth resource parameter to include during MCP login.
`mcp_servers.<id>.experimental_environment`	`local / remote`	Experimental placement for an MCP server. `remote` starts stdio servers through a remote executor environment; streamable HTTP...
`agents`	`table`	Multi-agent settings and custom role declarations.
`agents.enabled`	`boolean`	Enable or disable multi-agent tools (default: true).
`agents.max_concurrent_threads_per_session`	`number`	Maximum number of spawned-agent threads that can be open concurrently, excluding the primary thread.
`agents.max_threads`	`number`	Legacy alias for `agents.max_concurrent_threads_per_session`.
`agents.default_subagent_model`	`string`	Default model for spawned agents.
`agents.default_subagent_reasoning_effort`	`string`	Default reasoning effort for spawned agents.
`agents.interrupt_message`	`boolean`	Record a model-visible message when an agent turn is interrupted (default: true).
`agents.<name>.description`	`string`	Role guidance shown to Codex when choosing and spawning that agent type.
`agents.<name>.config_file`	`string (path)`	Path to a TOML config layer for that role; relative paths resolve from the config file that declares the role.
`memories.generate_memories`	`boolean`	When `false`, newly created threads are not stored as memory-generation inputs.
`memories.use_memories`	`boolean`	When `false`, Codex skips injecting existing memories into future sessions.

`memories.disable_on_external_context`	`boolean`	When `true`, threads that use external context such as MCP tool calls, web search, or tool search are kept out of memory...
`memories.max_raw_memories_for_consolidation`	`number`	Maximum recent raw memories retained for global consolidation.
`memories.max_unused_days`	`number`	Maximum days since a memory was last used before it becomes ineligible for consolidation.
`memories.max_rollout_age_days`	`number`	Maximum age of threads considered for memory generation.
`memories.max_rollouts_per_startup`	`number`	Maximum rollout candidates processed per startup pass.
`memories.min_rollout_idle_hours`	`number`	Minimum idle time before a thread is considered for memory generation.
`memories.min_rate_limit_remaining_percent`	`number`	Minimum remaining percentage required in Codex rate-limit windows before memory generation starts.
`memories.extract_model`	`string`	Optional model override for per-thread memory extraction.
`memories.consolidation_model`	`string`	Optional model override for global memory consolidation.
`features.prefer_mxc`	`boolean`	Prefer MXC for local Windows execution when native capabilities and policy allow it; otherwise retain the configured legacy...
`features.unified_exec`	`boolean`	Use resumable command execution (stable; enabled on all platforms).
`features.shell_snapshot`	`boolean`	Snapshot shell environment to speed up repeated commands (stable; on by default).
`features.multi_agent`	`boolean`	Enable multi-agent collaboration tools (`spawn_agent`, `send_input`, `resume_agent`, `wait_agent`, and `close_agent`) (stable;...
`features.goals`	`boolean`	Enable persisted goals and automatic continuation (stable; on by default).
`features.remote_plugin`	`boolean`	Enable the remote plugin catalog (stable; on by default).
`features.personality`	`boolean`	Removed feature flag.
`features.network_proxy`	`boolean / table`	Start the network proxy for sandboxed commands (experimental; off by default).
`features.network_proxy.enabled`	`boolean`	Start the sandboxed-command network proxy when command network access is enabled.
`features.network_proxy.domains`	`map<string, allow / deny>`	Domain policy for sandboxed networking.
`features.network_proxy.unix_sockets`	`map<string, allow / deny>`	Unix socket policy for sandboxed networking.
`features.network_proxy.allow_local_binding`	`boolean`	Allow broader local/private-network access.
`features.network_proxy.enable_socks5`	`boolean`	Expose SOCKS5 support.

`features.network_proxy.enable_socks5_udp`	`boolean`	Allow UDP over SOCKS5. Defaults to `true`.
`features.network_proxy.allow_upstream_proxy`	`boolean`	Allow chaining through an upstream proxy from the environment.
`features.network_proxy.dangerously_allow_non_loopback_proxy`	`boolean`	Permit non-loopback listener addresses.
`features.network_proxy.dangerously_allow_all_unix_sockets`	`boolean`	Permit arbitrary Unix socket destinations instead of allowlist-only access.
`features.web_search`	`boolean`	Deprecated legacy toggle; prefer the top-level `web_search` setting.
`features.shell_tool`	`boolean`	Enable command execution tools (stable; on by default).
`features.enable_request_compression`	`boolean`	Compress streaming request bodies with zstd when supported (stable; on by default).
`features.skill_mcp_dependency_install`	`boolean`	Allow prompting and installing missing MCP dependencies for skills (stable; on by default).
`features.fast_mode`	`boolean`	Enable Fast mode controls advertised by the active model (stable; on by default), independently of `features.ultrafast_mode`.
`features.ultrafast_mode`	`boolean`	Enable Ultrafast mode controls (stable; on by default), independently of `features.fast_mode`.
`features.prevent_idle_sleep`	`boolean`	Prevent the machine from sleeping while a turn is actively running (experimental; off by default).
`suppress_unstable_features_warning`	`boolean`	Suppress the warning that appears when under-development feature flags are enabled.
`model_providers.<id>`	`table`	Custom provider definition.
`model_providers.<id>.name`	`string`	Display name for a custom model provider.
`model_providers.<id>.base_url`	`string`	API base URL for the model provider.
`model_providers.<id>.model_catalog_url`	`string`	Full URL for a Codex-native model catalog.
`model_providers.<id>.capabilities.external_web_access`	`boolean`	Whether the provider's hosted web search can access the live web.

`model_providers.<id>.capabilities.remote_compaction`	`unsupported / v2`	Provider compaction protocol. `unsupported` uses local compaction; `v2` sends compaction triggers through Responses.
`model_providers.<id>.env_key`	`string`	Environment variable supplying the provider API key.
`model_providers.<id>.env_key_instructions`	`string`	Optional setup guidance for the provider API key.
`model_providers.<id>.experimental_bearer_token`	`string`	Direct bearer token for the provider (discouraged; use `env_key`).
`model_providers.<id>.requires_openai_auth`	`boolean`	The provider uses OpenAI authentication (defaults to false).
`model_providers.<id>.wire_api`	`responses`	Protocol used by the provider. `responses` is the only supported value, and it is the default when omitted.
`model_providers.<id>.query_params`	`map<string,string>`	Extra query parameters appended to provider requests.
`model_providers.<id>.http_headers`	`map<string,string>`	Static HTTP headers added to provider requests.
`model_providers.<id>.env_http_headers`	`map<string,string>`	HTTP headers populated from environment variables when present.
`model_providers.<id>.request_max_retries`	`number`	Retry count for HTTP requests to the provider (default: 4).
`model_providers.<id>.stream_max_retries`	`number`	Retry count for SSE streaming interruptions (default: 5).
`model_providers.<id>.stream_idle_timeout_ms`	`number`	Idle timeout for SSE streams in milliseconds (default: 300000).
`model_providers.<id>.supports_websockets`	`boolean`	Whether that provider supports the Responses API WebSocket transport.
`model_providers.<id>.supports_standalone_web_search`	`boolean`	Advertise support for a compatible standalone web search endpoint (default: false).
`model_providers.<id>.auth`	`table`	Command-backed bearer token configuration for a custom provider.
`model_providers.<id>.auth.command`	`string`	Command to run when Codex needs a bearer token.
`model_providers.<id>.auth.args`	`array<string>`	Arguments passed to the token command.
`model_providers.<id>.auth.timeout_ms`	`number`	Maximum token command runtime in milliseconds (default: 5000).
`model_providers.<id>.auth.refresh_interval_ms`	`number`	How often Codex proactively refreshes the token in milliseconds (default: 300000).
`model_providers.<id>.auth.cwd`	`string (path)`	Working directory for the token command.
`model_providers.amazon-bedrock.aws.profile`	`string`	AWS profile name used by the built-in `amazon-bedrock` provider.
`model_providers.amazon-bedrock.aws.region`	`string`	AWS region used by the built-in `amazon-bedrock` provider.

`model_reasoning_effort`	`string`	Reasoning effort advertised by the selected model, such as `low`, `medium`, `high`, `xhigh`, `max`, or `ultra`.
`plan_mode_reasoning_effort`	`string`	Plan-mode-specific reasoning override using a level supported by the selected model.
`model_reasoning_summary`	`auto / concise / detailed / none`	Select reasoning summary detail or disable summaries entirely.
`model_verbosity`	`low / medium / high`	Optional GPT-5 Responses API verbosity override; when unset, the selected model/preset default is used.
`model_supports_reasoning_summaries`	`boolean`	Force Codex to send or not send reasoning metadata.
`shell_environment_policy.inherit`	`all / core / none`	Baseline environment inheritance when spawning subprocesses.
`shell_environment_policy.ignore_default_excludes`	`boolean`	Keep variables containing KEY, SECRET, or TOKEN before other filters run (default: true).
`shell_environment_policy.filters`	`map<string, include / exclude>`	Canonical case-insensitive environment-variable pattern filters.
`shell_environment_policy.exclude`	`array<string>`	Legacy environment-variable exclusion patterns.
`shell_environment_policy.include_only`	`array<string>`	Legacy allowlist of environment-variable patterns.
`shell_environment_policy.set`	`map<string,string>`	Explicit environment values injected after exclusions; include filters can still remove them.
`shell_environment_policy.experimental_use_profile`	`boolean`	Use the user shell profile when spawning subprocesses.
`project_root_markers`	`array<string>`	List of project root marker filenames; used when searching parent directories for the project root.
`project_doc_max_bytes`	`number`	Maximum bytes read from `AGENTS.md` when building project instructions.
`project_doc_fallback_filenames`	`array<string>`	Additional filenames to try when `AGENTS.md` is missing.
`history.persistence`	`save-all / none`	Control whether Codex saves session transcripts to history.jsonl.
`tool_output_token_limit`	`number`	Token budget for storing individual tool/function outputs in history.

`background_terminal_max_timeout`	`number`	Maximum poll window in milliseconds for empty `write_stdin` polls (background terminal polling).
`history.max_bytes`	`number`	If set, caps the history file size in bytes by dropping oldest entries.
`file_opener`	`vscode / vscode-insiders / windsurf /…`	URI scheme used to open citations from Codex output (default: `vscode`).
`otel.environment`	`string`	Environment tag applied to emitted OpenTelemetry events (default: `dev`).
`otel.exporter`	`none / otlp-http / otlp-grpc`	Select the OpenTelemetry exporter and provide any endpoint metadata.
`otel.trace_exporter`	`none / otlp-http / otlp-grpc`	Select the OpenTelemetry trace exporter and provide any endpoint metadata.
`otel.metrics_exporter`	`none / statsig / otlp-http / otlp-grpc`	Select the OpenTelemetry metrics exporter (defaults to `statsig`).
`otel.log_user_prompt`	`boolean`	Opt in to exporting raw user prompts with OpenTelemetry logs.
`otel.exporter.<id>.endpoint`	`string`	Exporter endpoint for OTEL logs.
`otel.exporter.<id>.protocol`	`binary / json`	Protocol used by the OTLP/HTTP exporter.
`otel.exporter.<id>.headers`	`map<string,string>`	Static headers included with OTEL exporter requests.
`otel.trace_exporter.<id>.endpoint`	`string`	Trace exporter endpoint for OTEL logs.
`otel.trace_exporter.<id>.protocol`	`binary / json`	Protocol used by the OTLP/HTTP trace exporter.
`otel.trace_exporter.<id>.headers`	`map<string,string>`	Static headers included with OTEL trace exporter requests.
`otel.exporter.<id>.tls.ca-certificate`	`string`	CA certificate path for OTEL exporter TLS.
`otel.exporter.<id>.tls.client-certificate`	`string`	Client certificate path for OTEL exporter TLS.
`otel.exporter.<id>.tls.client-private-key`	`string`	Client private key path for OTEL exporter TLS.
`otel.trace_exporter.<id>.tls.ca-certificate`	`string`	CA certificate path for OTEL trace exporter TLS.
`otel.trace_exporter.<id>.tls.client-certificate`	`string`	Client certificate path for OTEL trace exporter TLS.
`otel.trace_exporter.<id>.tls.client-private-key`	`string`	Client private key path for OTEL trace exporter TLS.
`desktop.custom_file_handlers.<id>`	`table`	User-level only.
`desktop.custom_file_handlers.<id>.label`	`string`	Display name shown in **Open in** menus.
`desktop.custom_file_handlers.<id>.icon`	`string`	Bundled asset path, Base64-encoded `data:image/...` URL, file URI, or absolute local path for the handler icon.

`desktop.custom_file_handlers.<id>.command`	`string`	Executable path or command name to detect and launch.
`desktop.custom_file_handlers.<id>.args`	`array<string>`	Arguments inserted between the command and file input (default: `[]`).
`desktop.custom_file_handlers.<id>.input`	`path / json_argument / json_stdin`	How the app sends file input to the handler (default: `path`).
`desktop.custom_file_handlers.<id>.supports_ssh`	`boolean`	Offer the handler for files in SSH workspaces (default: `false`).
`tui`	`table`	TUI-specific options such as enabling inline desktop notifications.
`tui.notifications`	`boolean / array<string>`	Enable TUI notifications; optionally restrict to specific event types.
`tui.notification_method`	`auto / osc9 / bel`	Notification method for terminal notifications (default: auto).
`tui.notification_condition`	`unfocused / always`	Control whether TUI notifications fire only when the terminal is unfocused or regardless of focus.
`tui.animations`	`boolean`	Enable terminal animations (welcome screen, shimmer, spinner) (default: true).
`tui.disable_paste_burst`	`boolean`	Disable burst-paste detection in the TUI (default: false).
`tui.alternate_screen`	`auto / always / never`	Control alternate screen usage for the TUI (default: auto; auto skips it in Zellij to preserve scrollback).
`tui.resume_cwd`	`current / session`	Working directory to use when resuming or forking a session.
`tui.vim_mode_default`	`boolean`	Start the composer in Vim normal mode instead of insert mode (default: false).
`tui.raw_output_mode`	`boolean`	Start the TUI in raw scrollback mode for copy-friendly terminal selection (default: false).
`tui.show_tooltips`	`boolean`	Show onboarding tooltips in the TUI welcome screen (default: true).
`tui.auto_recap`	`boolean`	Generate automatic conversation recaps while the terminal is unfocused (default: true).
`tui.show_server_version_notice`	`boolean`	Show informational notices about connected app-server version differences (default: true).
`tui.status_line`	`array<string> / null`	Ordered list of TUI footer status-line item identifiers. `null` disables the status line.
`tui.theme`	`string`	Syntax-highlighting theme override (kebab-case theme name).
`tui.keymap.<context>.<action>`	`string / array<string>`	Keyboard shortcut binding for a TUI action.
`tui.keymap.<context>.<action> = []`	`empty array`	Unbind the action in that keymap context.
`marketplaces.<name>.source_type`	`git / local`	Source kind for a configured plugin marketplace.

`marketplaces.<name>.source`	`string`	Git repository location or local marketplace root directory.
`marketplaces.<name>.ref`	`string`	Optional Git branch, tag, or commit for the marketplace.
`marketplaces.<name>.sparse_paths`	`array<string>`	Optional sparse checkout paths for a Git marketplace.
`plugins.<plugin>.enabled`	`boolean`	Enable or disable a local-marketplace plugin using a `plugin-name@marketplace-name` key.
`plugins.<plugin>.mcp_servers.<server>.enabled`	`boolean`	Enable or disable an MCP server bundled by an installed plugin without changing the plugin manifest.
`plugins.<plugin>.mcp_servers.<server>.default_tools_approval_mode`	`auto / prompt / writes / approve`	Default approval behavior for tools on a plugin-provided MCP server.
`plugins.<plugin>.mcp_servers.<server>.enabled_tools`	`array<string>`	Allow list of tools exposed from a plugin-provided MCP server.
`plugins.<plugin>.mcp_servers.<server>.disabled_tools`	`array<string>`	Deny list applied after `enabled_tools` for a plugin-provided MCP server.
`plugins.<plugin>.mcp_servers.<server>.tools.<tool>.approval_mode`	`auto / prompt / writes / approve`	Per-tool approval behavior override for a plugin-provided MCP tool.
`tui.model_availability_nux.<model>`	`integer`	Internal startup-tooltip state keyed by model slug.
`hide_agent_reasoning`	`boolean`	Suppress reasoning events in both the TUI and `codex exec` output.
`show_raw_agent_reasoning`	`boolean`	Surface raw reasoning content when the active model emits it.
`disable_paste_burst`	`boolean`	Legacy fallback for `tui.disable_paste_burst`.
`windows_wsl_setup_acknowledged`	`boolean`	Track Windows onboarding acknowledgement (Windows only).
`chatgpt_base_url`	`string`	Override the base URL used during the ChatGPT login flow.
`cli_auth_credentials_store`	`file / keyring / auto / ephemeral`	Control where the CLI stores cached credentials.
`mcp_oauth_credentials_store`	`auto / file / keyring`	Preferred store for MCP OAuth credentials.
`mcp_oauth_callback_port`	`integer`	Optional global fixed port for the local HTTP callback server used during MCP OAuth login.
`mcp_oauth_callback_url`	`string`	Optional base callback URL for MCP OAuth login, such as a devbox ingress URL. Newly added pre-registered clients use this URL...
`experimental_use_unified_exec_tool`	`boolean`	Legacy name for enabling unified exec; prefer `[features].unified_exec` or `codex --enable unified_exec`.
`tools.view_image`	`boolean`	Enable the local-image attachment tool `view_image`.

`default_permissions`	`string`	Name of the default permissions profile to apply to sandboxed tool calls.
`permissions.<name>.description`	`string`	Human-readable description for this named profile.
`permissions.<name>.extends`	`string`	Optional parent profile applied before this named profile.
`permissions.<name>.workspace_roots`	`table`	Profile-defined workspace roots that receive `:workspace_roots` filesystem rules alongside the session's runtime workspace roots.
`permissions.<name>.workspace_roots.<path>`	`boolean`	Opt a path into the profile's workspace root set when `true`.
`permissions.<name>.filesystem`	`table`	Named filesystem permission profile.
`permissions.<name>.filesystem.glob_scan_max_depth`	`number`	Maximum depth for expanding deny-read glob patterns on platforms that snapshot matches before sandbox startup.
`permissions.<name>.network.enabled`	`boolean`	Enable network access for commands in this permission profile.
`permissions.<name>.network.proxy_url`	`string`	HTTP listener URL used when this permissions profile enables sandboxed networking.
`permissions.<name>.network.enable_socks5`	`boolean`	Expose SOCKS5 support when this permissions profile enables sandboxed networking.
`permissions.<name>.network.socks_url`	`string`	SOCKS5 proxy endpoint used by this permissions profile.
`permissions.<name>.network.enable_socks5_udp`	`boolean`	Allow UDP over the SOCKS5 listener when enabled.
`permissions.<name>.network.allow_upstream_proxy`	`boolean`	Allow sandboxed networking to chain through another upstream proxy.
`permissions.<name>.network.dangerously_allow_non_loopback_proxy`	`boolean`	Permit non-loopback bind addresses for sandboxed networking listeners.
`permissions.<name>.network.dangerously_allow_all_unix_sockets`	`boolean`	Allow arbitrary Unix socket destinations instead of the default restricted set.
`permissions.<name>.network.mode`	`limited / full`	Network proxy mode used for subprocess traffic.
`permissions.<name>.network.domains`	`table`	Domain rules for sandboxed commands.
`permissions.<name>.network.domains.<pattern>`	`allow / deny`	Allow or deny an exact host or scoped wildcard pattern such as `*.example.com` or `**.example.com`.
`permissions.<name>.network.unix_sockets`	`table`	Unix socket allowlist overrides for sandboxed networking.
`permissions.<name>.network.unix_sockets.<path>`	`allow / deny`	Add an absolute Unix socket path to the effective allowlist with `allow`, or reject it with `deny`.
`permissions.<name>.network.allow_local_binding`	`boolean`	Permit broader local/private-network access through sandboxed networking.

`notice.hide_full_access_warning`	`boolean`	Track acknowledgement of the full access warning prompt.
`notice.hide_world_writable_warning`	`boolean`	Track acknowledgement of the Windows world-writable directories warning.
`notice.hide_rate_limit_model_nudge`	`boolean`	Track opt-out of the rate limit model switch reminder.
`notice.hide_gpt5_1_migration_prompt`	`boolean`	Track acknowledgement of the GPT-5.1 migration prompt.
`notice.hide_gpt-5.1-codex-max_migration_prompt`	`boolean`	Track acknowledgement of the gpt-5.1-codex-max migration prompt.
`notice.model_migrations`	`map<string,string>`	Track acknowledged model migrations as old->new mappings.
`forced_login_method`	`chatgpt / api`	Restrict Codex to a specific authentication method.
`forced_chatgpt_workspace_id`	`string (uuid)`	Limit ChatGPT logins to a specific workspace identifier.

16.3 requirements.toml keys (174)

Admins use `requirements.toml` to enforce policy that users can't override. For example, `allow_managed_hooks_only = true` ignores user, project and session hooks, and works only in this file.

KEY	TYPE	WHAT IT DOES
`sqlite_home`	`string (path)`	Enforce the directory where Codex stores SQLite-backed runtime state.
`log_dir`	`string (path)`	Enforce the directory where Codex writes local log files.
`model_catalog_json`	`string (path)`	Enforce the JSON model catalog Codex uses at startup.
`check_for_update_on_startup`	`boolean`	Enforce whether Codex checks for updates when it starts.
`allow_login_shell`	`boolean`	Enforce whether shell tools can start a login shell.
`allowed_login_methods`	`array<string>`	Allow `chatgpt`, `api`, or both.
`allowed_chatgpt_workspaces`	`array<string>`	Restrict ChatGPT login, including Codex access tokens, to the listed workspace IDs.
`cli_auth_credentials_store`	`file / keyring / auto / ephemeral`	Enforce the CLI credential store before authentication loads. `file` uses `CODEX_HOME/auth.json`; `keyring` requires the OS...
`chatgpt_base_url`	`string`	Enforce the ChatGPT service base URL before authentication and cloud-policy retrieval.
`feedback`	`table`	Managed feedback settings.
`feedback.enabled`	`boolean`	Enforce whether users can submit feedback across Codex clients.
`allowed_approval_policies`	`array<string>`	Allowed approval policies, such as `on-request`, `never`, and `granular`.

`allowed_approvals_reviewers`	`array<string>`	Allowed values for `approvals_reviewer`, such as `user` and `auto_review`.
`guardian_policy_config`	`string`	Managed Markdown policy instructions for automatic review.
`guardian_extra_policy`	`string`	Additional managed Markdown policy for automatic review, included alongside the main policy.
`additional_developer_instructions`	`string`	Managed developer instructions added as a separate developer message.
`auto_review`	`table`	Managed automatic-review requirements.
`auto_review.required_on_models`	`array<string>`	Model slugs that must use automatic review.
`auto_review.ignore_rules`	`array<string>`	Full model slugs for which Codex ignores `allow` prefix rules in command execution policy.
`allowed_permission_profiles`	`table<boolean>`	Complete list of allowed permission profiles.
`allowed_permission_profiles.<name>`	`boolean`	Allow or deny a built-in or custom permission profile defined in a loaded config or requirements source.
`default_permissions`	`string`	Managed default permission profile.
`enforce_residency`	`string`	Require Codex service traffic to use a supported data residency.
`model_provider`	`string`	Enforce the model provider ID, overriding local and session configuration.
`model_providers`	`map<string, table>`	Managed model provider definitions.
`model_providers.<id>`	`table`	Complete managed provider definition.
`models`	`table`	Contains the `[models.new_thread]` table.
`models.new_thread`	`table`	Optional defaults to apply when a new local thread starts.
`models.new_thread.model`	`string`	Default model for new threads.
`models.new_thread.model_reasoning_effort`	`string`	Default reasoning effort for new threads.
`models.new_thread.service_tier`	`string`	Default service tier for new threads.
`permissions`	`table`	Admin-defined permission profiles keyed by profile name.
`permissions.<name>`	`table`	Admin-defined permission profile.
`allowed_sandbox_modes`	`array<string>`	Allowed values for `sandbox_mode`.
`windows`	`table`	Native Windows sandbox requirements.
`windows.allowed_sandbox_implementations`	`array<string>`	Allowed legacy native Windows sandbox implementations (`elevated` and `unelevated`).
`windows.allow_mxc`	`boolean`	Set to `false` to prohibit both explicit MXC selection and automatic selection through `features.prefer_mxc`.
`remote_sandbox_config`	`array<table>`	Host-specific sandbox requirements.

`remote_sandbox_config[].hostname_patterns`	`array<string>`	Case-insensitive host name patterns.
`remote_sandbox_config[].allowed_sandbox_modes`	`array<string>`	Allowed sandbox modes to apply when this host-specific entry matches.
`allowed_web_search_modes`	`array<string>`	Allowed values for `web_search` (`disabled`, `cached`, `indexed`, `live`). `disabled` is always allowed; an empty list...
`allow_managed_hooks_only`	`boolean`	When `true`, Codex skips user, project, session, and plugin hooks while still allowing managed hooks from `requirements.toml`...
`allow_appshots`	`boolean`	Set to `false` to disable Appshots for managed users.
`allow_remote_control`	`boolean`	Set to `false` to disable device remote control for managed users.
`allow_browser_and_computer_use`	`boolean`	Set to `false` to block both agent-driven Browser Use and native-app Computer Use.
`features.plugin_sharing`	`boolean`	Set to `false` in cloud-managed `requirements.toml` to disable workspace sharing for locally built plugins.
`features`	`table`	Pinned feature values.
`features.<name>`	`boolean`	Require a documented runtime or app feature to stay enabled or disabled.
`features.apps`	`boolean`	Pin Apps integration availability on or off for managed users.
`features.in_app_updates`	`boolean`	Set to `false` in `requirements.toml` to disable in-app updates.
`features.in_app_browser`	`boolean`	Set to `false` in `requirements.toml` to disable the built-in browser pane that users open and control directly.
`features.in_app_chat`	`boolean`	Set to `false` to hide ChatGPT and ChatGPT Work conversation screens and related cloud automation UI in the ChatGPT desktop app.
`features.in_app_dictation`	`boolean`	Set to `false` to disable in-app dictation in the desktop app.
`features.in_app_local_automation`	`boolean`	Set to `false` to disable local scheduled tasks in the desktop app.
`features.browser_use`	`boolean`	Set to `false` in `requirements.toml` to disable agent-driven Browser Use.
`features.browser_use_external`	`boolean`	Set to `false` in `requirements.toml` to prevent Codex from operating supported browsers through the ChatGPT browser...
`features.browser_use_full_cdp_access`	`boolean`	Set to `false` in `requirements.toml` to disable full Chrome DevTools Protocol access in the local runtime, including Browser...
`features.fast_mode`	`boolean`	Pin the canonical `fast_mode` feature on or off for managed users.
`features.guardian_approval`	`boolean`	Pin Guardian approval availability on or off for managed users.

`features.memories`	`boolean`	Pin Memories availability on or off for managed users.
`features.multi_agent`	`boolean`	Pin multi-agent availability on or off for managed users.
`features.plugins`	`boolean`	Pin plugin availability on or off for managed users.
`features.remote_plugin`	`boolean`	Pin remote plugin catalog availability on or off for managed users.
`features.realtime_conversation`	`boolean`	Set to `false` to disable the experimental `/voice` command in the Codex CLI. Do not rely on this setting to block ChatGPT...
`features.computer_use`	`boolean`	Set to `false` in `requirements.toml` to disable Computer Use, Record & Replay, and related install or enablement flows.
`features.workspace_dependencies`	`boolean`	Pin bundled workspace-dependency runtime availability on or off for managed users.
`in_app_browser`	`table`	Requirements for the built-in browser pane.
`in_app_browser.allow_external_browser_settings_import`	`boolean`	Set to `false` to prevent users from importing settings or browsing data from an external browser into the built-in browser.
`browser_use`	`table`	Managed requirements for agent-driven Browser Use.
`browser_use.allow_webmcp`	`boolean`	Set to `true` to opt the desktop app's built-in browser into site tools (WebMCP).
`browser_use.allow_history_access`	`boolean`	Set to `false` to prevent Browser Use from reading browser history.
`browser_use.disable_auto_review`	`boolean`	Set to `true` to skip automatic review for Browser Use and ask the user for approval instead.
`browser_use.allow_global_persistent_approval`	`boolean`	Set to `false` to prevent Browser Use from creating or honoring `Always allow` approvals that cover every site, such as...
`browser_use.default_origin_policy`	`table`	Fallback for each Browser Use setting when no matching entry under `browser_use.origins` defines it.
`browser_use.default_origin_policy.access`	`allow / deny`	Use `deny` to block Browser Use on origins that use the fallback.
`browser_use.default_origin_policy.downloads`	`allow / deny`	Use `deny` to block Browser Use downloads on origins that use the fallback. `allow` only lets normal approval and policy...
`browser_use.default_origin_policy.uploads`	`allow / deny`	Use `deny` to block Browser Use uploads on origins that use the fallback. `allow` only lets normal approval and policy checks...
`browser_use.default_origin_policy.full_cdp_access`	`allow / deny`	Use `deny` to block full Chrome DevTools Protocol (CDP) access on origins that use the fallback. `allow` only lets normal...

`browser_use.default_origin_policy.auto_review`	`allow / deny`	Use `deny` to skip automatic review on origins that use the fallback and ask the user for approval instead. `allow` leaves...
`browser_use.default_origin_policy.persistent_approval`	`boolean`	Set to `false` to prevent Browser Use from saving or honoring an `Always allow` approval on origins that use the fallback.
`browser_use.default_origin_policy.access_approval_lifetime`	`turn / thread`	Set how long a non-persistent site-access approval lasts: `turn` limits it to the current turn, and `thread` keeps it for the...
`browser_use.origins.<pattern>`	`table`	Policy for origins matching this pattern.
`browser_use.origins.<pattern>.access`	`allow / deny`	Use `deny` to block Browser Use on matching origins.
`browser_use.origins.<pattern>.downloads`	`allow / deny`	Use `deny` to block Browser Use downloads on matching origins. `allow` only lets normal approval and policy checks continue.
`browser_use.origins.<pattern>.uploads`	`allow / deny`	Use `deny` to block Browser Use uploads on matching origins. `allow` only lets normal approval and policy checks continue.
`browser_use.origins.<pattern>.full_cdp_access`	`allow / deny`	Use `deny` to block full Chrome DevTools Protocol (CDP) access on matching origins. `allow` only lets normal opt-in and...
`browser_use.origins.<pattern>.auto_review`	`allow / deny`	Use `deny` to skip automatic review on matching origins and ask the user for approval instead. `allow` leaves automatic review...
`browser_use.origins.<pattern>.persistent_approval`	`boolean`	Set to `false` to prevent Browser Use from saving or honoring an `Always allow` approval on matching origins.
`browser_use.origins.<pattern>.access_approval_lifetime`	`turn / thread`	Set how long a non-persistent site-access approval for matching origins lasts: `turn` limits it to the current turn, and...
`computer_use`	`table`	Managed requirements for agent-driven work in native desktop apps.
`computer_use.allow_locked_computer_use`	`boolean`	Set to `false` to prevent users from enabling Locked Use on a managed macOS device.
`computer_use.allow_persistent_approval`	`boolean`	Set to `false` to remove the option to save app approvals across sessions.

`computer_use.default_app_access`	`allow / deny`	Fallback access for native apps that do not match a platform-specific rule. `deny` blocks access. `allow` only lets normal...
`computer_use.macos`	`table`	Computer Use app rules for macOS.
`computer_use.macos.bundle_ids`	`map<string, allow / deny>`	Map exact macOS bundle identifiers to `allow` or `deny`.
`computer_use.macos.bundle_ids.<bundle-id>`	`allow / deny`	Use `deny` to block the exact bundle identifier. `allow` overrides only this policy source's default and still requires any...
`computer_use.windows`	`table`	Computer Use app rules for packaged and unpackaged Windows apps.
`computer_use.windows.aumids`	`map<string, allow / deny>`	Map exact, registered Application User Model IDs (AUMIDs) for signed packaged apps to `allow` or `deny`.
`computer_use.windows.aumids.<aumid>`	`allow / deny`	Use `deny` to block the exact packaged-app identity. `allow` overrides only this policy source's default and still requires...
`computer_use.windows.exes`	`array<table>`	Rules for signed, unpackaged Windows executables.
`computer_use.windows.exes[].publisher_name`	`string`	Required exact publisher name from the executable's trusted signing certificate, formatted as a Windows X.500 distinguished name.
`computer_use.windows.exes[].product_name`	`string`	Required exact `ProductName` from the executable's signed version information.
`computer_use.windows.exes[].binary_name`	`string`	Optional `OriginalFilename` from the executable's signed version information.
`computer_use.windows.exes[].access`	`allow / deny`	Required access decision for matching executables. `deny` blocks access. `allow` overrides only this policy source's default...
`application`	`table`	Managed desktop application requirements.
`application.network`	`table`	Destination restrictions for desktop app network requests, separate from command networking and browser origin rules.
`application.network.enabled`	`boolean`	Defaults to true when the table is present.
`application.network.domains`	`table<allow / deny>`	Exact domain rules for desktop app requests.
`application.network.domains.<domain>`	`allow / deny`	Allow or deny an exact domain.
`experimental_network`	`table`	Administrator-managed network requirements for sandboxed local commands, enforced from `requirements.toml`.
`experimental_network.enabled`	`boolean`	Enable sandboxed networking requirements.
`experimental_network.http_port`	`integer`	Loopback HTTP listener port to use for `[experimental_network]` requirements.
`experimental_network.socks_port`	`integer`	Loopback SOCKS5 listener port to use for `[experimental_network]` requirements.

`experimental_network.allow_upstream_proxy`	`boolean`	Allow sandboxed networking to chain through an upstream proxy from the environment.
`experimental_network.dangerously_allow_non_loopback_proxy`	`boolean`	Permit non-loopback listener addresses for `[experimental_network]` requirements.
`experimental_network.dangerously_allow_all_unix_sockets`	`boolean`	Permit arbitrary Unix socket destinations instead of allowlist-only access.
`experimental_network.domains`	`map<string, allow / deny>`	Map-shaped administrator domain policy for sandboxed networking.
`experimental_network.domains.<pattern>`	`allow / deny`	Allow or deny sandboxed network access for the matching domain pattern.
`experimental_network.allowed_domains`	`array<string>`	Administrator allow rules for sandboxed-command networking while the managed network proxy is enabled.
`experimental_network.denied_domains`	`array<string>`	List-shaped administrator deny rules for sandboxed networking.
`experimental_network.managed_allowed_domains_only`	`boolean`	When networking requirements are enabled and this is true, ordinary user configuration and per-domain approvals cannot expand…
`experimental_network.unix_sockets`	`map<string, allow / deny>`	Administrator-managed Unix socket allowlist for sandboxed networking on macOS. Paths must be absolute.
`experimental_network.unix_sockets.<path>`	`allow / deny`	On macOS, `allow` adds an absolute Unix socket path to the allowlist; `deny` leaves it out.
`experimental_network.allow_local_binding`	`boolean`	Permit broader local/private-network access for sandboxed networking.
`hooks`	`table`	Admin-enforced managed lifecycle hooks.
`hooks.managed_dir`	`string (absolute path)`	Directory containing managed hook scripts on macOS and Linux.
`hooks.windows_managed_dir`	`string (absolute path)`	Directory containing managed hook scripts on Windows.
`hooks.<Event>`	`array<table>`	Matcher groups for a hook event such as `PreToolUse`, `PermissionRequest`, `PostToolUse`, `PreCompact`, `PostCompact`,…

`hooks.<Event>[].hooks`	`array<table>`	Hook handlers for a matcher group.
`hooks.<Event>[].hooks[].async`	`boolean`	Run a command hook in the background without delaying the triggering operation.
`hooks.<Event>[].hooks[].additionalContextLimit`	`integer`	Approximate per-handler token threshold for saving oversized `additionalContext` to disk and showing the model a shorter preview.
`hooks.<Event>[].hooks[].commandWindows`	`string`	Windows-only command override for command hooks.
`permissions.filesystem.deny_read`	`array<string>`	Admin-enforced filesystem read denials.
`mcp_servers`	`table`	Allowlist of MCP servers that may be enabled.
`mcp_servers.<id>.identity`	`table`	Identity rule for a single MCP server.
`mcp_servers.<id>.identity.command`	`string / table`	Allow an MCP stdio server by exact command string, or use a matcher table to require an exact executable and ordered argument...
`mcp_servers.<id>.identity.command.executable`	`string`	Executable that the stdio server's configured `command` must match exactly.
`mcp_servers.<id>.identity.command.args`	`array<table>`	Ordered argument matchers for a stdio server.
`mcp_servers.<id>.identity.command.args[].match`	`exact / prefix / regex`	Match operation for this argument position.
`mcp_servers.<id>.identity.command.args[].value`	`string`	Value used by an `exact` or `prefix` argument matcher.
`mcp_servers.<id>.identity.command.args[].expression`	`string`	Regular expression used by a `regex` argument matcher.
`mcp_servers.<id>.identity.url`	`string / table`	Allow an MCP streamable HTTP server by exact URL string, or use an `exact`, `prefix`, or `regex` value matcher table.
`mcp_servers.<id>.identity.url.match`	`exact / prefix / regex`	Match operation for the configured MCP server URL.
`mcp_servers.<id>.identity.url.value`	`string`	Value used by an `exact` or `prefix` URL matcher.
`mcp_servers.<id>.identity.url.expression`	`string`	Regular expression used by a `regex` URL matcher.
`plugins`	`table`	Plugin-specific MCP server allowlists keyed by plugin identifier.
`plugins.<plugin>.mcp_servers`	`table`	Allowlist for MCP servers bundled with one plugin.
`plugins.<plugin>.mcp_servers.<server>.identity`	`table`	Identity rule for one plugin-bundled MCP server.
`plugins.<plugin>.mcp_servers.<server>.identity.command`	`string / table`	Allow a plugin's stdio MCP server by exact command string, or use a matcher table to require an exact executable and ordered...
`plugins.<plugin>.mcp_servers.<server>.identity.command.executable`	`string`	Executable that the plugin-bundled stdio server's configured command must match exactly.

`plugins.<plugin>.mcp_servers.<server>.identity.command.args`	`array<table>`	Ordered argument matchers for a plugin-bundled stdio server.
`plugins.<plugin>.mcp_servers.<server>.identity.command.args[].match`	`exact / prefix / regex`	Match operation for this argument position.
`plugins.<plugin>.mcp_servers.<server>.identity.command.args[].value`	`string`	Value used by an `exact` or `prefix` argument matcher.
`plugins.<plugin>.mcp_servers.<server>.identity.command.args[].expression`	`string`	Regular expression used by a `regex` argument matcher.
`plugins.<plugin>.mcp_servers.<server>.identity.url`	`string / table`	Allow a plugin's streamable HTTP MCP server by exact URL string, or use an `exact`, `prefix`, or `regex` value matcher table.
`plugins.<plugin>.mcp_servers.<server>.identity.url.match`	`exact / prefix / regex`	Match operation for the plugin-bundled MCP server URL.
`plugins.<plugin>.mcp_servers.<server>.identity.url.value`	`string`	Value used by an `exact` or `prefix` URL matcher.
`plugins.<plugin>.mcp_servers.<server>.identity.url.expression`	`string`	Regular expression used by a `regex` URL matcher.
`marketplaces`	`table`	Admin requirements for plugin marketplace sources.
`marketplaces.restrict_to_allowed_sources`	`boolean`	When `true`, require configured marketplace sources to match `allowed_sources` for marketplace add, plugin install, refresh,...
`marketplaces.allowed_sources`	`table`	Allowed marketplace sources keyed by administrator-chosen rule name.
`marketplaces.allowed_sources.<name>`	`table`	One allowed source rule.
`marketplaces.allowed_sources.<name>.source`	`git / host_pattern / local`	Marketplace source matcher type.
`marketplaces.allowed_sources.<name>.ref`	`string`	Optional exact Git ref for a `git` rule.
`apps`	`table`	Managed app requirements keyed by app identifier.
`apps.<id>.enabled`	`boolean`	Set to `false` to disable an app.
`apps.<id>.tools.<tool>.approval_mode`	`auto / prompt / writes / approve`	Set the managed approval mode for one app tool.
`rules`	`table`	Admin-enforced command rules merged with `.rules` files.
`rules.prefix_rules`	`array<table>`	List of enforced prefix rules.
`rules.prefix_rules[].pattern`	`array<table>`	Command prefix expressed as pattern tokens.
`rules.prefix_rules[].pattern[].token`	`string`	A single literal token at this position.
`rules.prefix_rules[].pattern[].any_of`	`array<string>`	A list of allowed alternative tokens at this position.

`rules.prefix_rules[].decision`	`prompt / forbidden`	Required.
`rules.prefix_rules[].justification`	`string`	Optional non-empty rationale surfaced in approval prompts or rejection messages.

TIP

You rarely need most of these keys. A practical starting point is to set only the approval policy, the sandbox mode and a few prefix rules for commands you run all the time, then add other settings when a specific need comes up. Re-check the official Codex configuration reference after each Codex update, because keys are added and renamed.

CHAPTER 17

Gemini CLI settings.json Reference

This chapter lists every setting in the Gemini CLI's `settings.json`. It was generated on October 16, 2026 from `docs/reference/configuration.md` in the google-gemini/gemini-cli repository, and descriptions are shortened.

- **User:** `~/.gemini/settings.json`. **Project:** `.gemini/settings.json`. System and admin files override both.
- `general.defaultApprovalMode` accepts `"default"`, `"auto_edit"` or `"plan"` only. **YOLO can only be turned on from the command line** (`--yolo` or `--approval-mode=yolo`).
- `security.disableYoloMode` and `admin.secureModeEnabled` both block YOLO.

SETTING	TYPE	DEFAULT	WHAT IT DOES
`policyPaths`	array	`[]`	Additional policy files or directories to load.
`adminPolicyPaths`	array	`[]`	Additional admin policy files or directories to load.
`general.preferredEditor`	enum	`undefined`	The preferred editor to open files in.
`general.openEditorInNewWindow`	boolean	`false`	Open VS Code-family editors in a new window when editing files.
`general.vimMode`	boolean	`false`	Enable Vim keybindings.
`general.defaultApprovalMode`	enum	`"default"`	The default approval mode for tool execution. 'default' prompts for approval, 'auto_edit' auto-approves edit tools,...
`general.devtools`	boolean	`false`	Enable DevTools inspector on launch.
`general.enableAutoUpdate`	boolean	`true`	Enable automatic updates.
`general.enableAutoUpdateNotification`	boolean	`true`	Enable update notification prompts.
`general.enableNotifications`	boolean	`false`	Enable terminal run-event notifications for action-required prompts and session completion.
`general.notificationMethod`	enum	`"auto"`	How to send terminal notifications.

`general.checkpointing.enabled`	boolean	`false`	Enable session checkpointing for recovery.
`general.plan.enabled`	boolean	`true`	Enable Plan Mode for read-only safety during planning.
`general.plan.directory`	string	`undefined`	The directory where planning artifacts are stored.
`general.plan.modelRouting`	boolean	`true`	Automatically switch between Pro and Flash models based on Plan Mode status.
`general.retryFetchErrors`	boolean	`true`	Retry on “exception TypeError: fetch failed sending request” errors.
`general.maxAttempts`	number	`10`	Maximum number of attempts for requests to the main chat model.
`general.debugKeystrokeLogging`	boolean	`false`	Enable debug logging of keystrokes to the console.
`general.sessionRetention.enabled`	boolean	`true`	Enable automatic session cleanup.
`general.sessionRetention.maxAge`	string	`"30d"`	Automatically delete chats older than this time period (e.g., “30d”, “7d”, “24h”, “1w”).
`general.sessionRetention.maxCount`	number	`undefined`	Alternative: Maximum number of sessions to keep (most recent).
`general.sessionRetention.minRetention`	string	`"1d"`	Minimum retention period (safety limit, defaults to “1d”).
`general.topicUpdateNarration`	boolean	`true`	Enable the Topic & Update communication model for reduced chattiness and structured progress reporting.
`general.logRagSnippets`	boolean	`false`	Log full Code Customization (RAG) retrieved snippets to a local file for debugging.
`output.format`	enum	`"text"`	The format of the CLI output.
`ui.debugRainbow`	boolean	`false`	Enable debug rainbow rendering.
`ui.theme`	string	`undefined`	The color theme for the UI. See the CLI themes guide for available options.
`ui.autoThemeSwitching`	boolean	`true`	Automatically switch between default light and dark themes based on terminal background color.
`ui.terminalBackgroundPollingInterval`	number	`60`	Interval in seconds to poll the terminal background color.
`ui.customThemes`	object	`{}`	Custom theme definitions.
`ui.hideWindowTitle`	boolean	`false`	Hide the window title bar.
`ui.inlineThinkingMode`	enum	`"off"`	Display model thinking inline: off or full.
`ui.showStatusInTitle`	boolean	`false`	Show Gemini CLI model thoughts in the terminal window title during the working phase.
`ui.dynamicWindowTitle`	boolean	`true`	Update the terminal window title with current status icons (Ready: ◇, Action Required: ✋, Working: ✦).
`ui.showHomeDirectoryWarning`	boolean	`true`	Show a warning when running Gemini CLI in the home directory.
`ui.showCompatibilityWarnings`	boolean	`true`	Show warnings about terminal or OS compatibility issues.
`ui.hideTips`	boolean	`false`	Hide helpful tips in the UI.

`ui.escapePastedAtSymbols`	boolean	`false`	When enabled, @ symbols in pasted text are escaped to prevent unintended @path expansion.
`ui.showShortcutsHint`	boolean	`true`	Show the "? for shortcuts" hint above the input.
`ui.compactToolOutput`	boolean	`true`	Display tool outputs (like directory listings and file reads) in a compact, structured format.
`ui.hideBanner`	boolean	`false`	Hide the application banner.
`ui.hideContextSummary`	boolean	`false`	Hide the context summary (GEMINI.md, MCP servers) above the input.
`ui.footer.items`	array	`undefined`	List of item IDs to display in the footer.
`ui.footer.showLabels`	boolean	`true`	Display a second line above the footer items with descriptive headers (e.g., /model).
`ui.footer.hideCWD`	boolean	`false`	Hide the current working directory in the footer.
`ui.footer.hideSandboxStatus`	boolean	`false`	Hide the sandbox status indicator in the footer.
`ui.footer.hideModelInfo`	boolean	`false`	Hide the model name and context usage in the footer.
`ui.footer.hideContextPercentage`	boolean	`true`	Hides the context window usage percentage.
`ui.hideFooter`	boolean	`false`	Hide the footer from the UI.
`ui.collapseDrawerDuringApproval`	boolean	`true`	Whether to collapse the UI drawer when a tool is awaiting confirmation.
`ui.showMemoryUsage`	boolean	`false`	Display memory usage information in the UI.
`ui.showLineNumbers`	boolean	`true`	Show line numbers in the chat.
`ui.showCitations`	boolean	`false`	Show citations for generated text in the chat.
`ui.showModelInfoInChat`	boolean	`false`	Show the model name in the chat for each model turn.
`ui.showUserIdentity`	boolean	`true`	Show the signed-in user's identity (e.g. email) in the UI.
`ui.useAlternateBuffer`	boolean	`false`	Use an alternate screen buffer for the UI, preserving shell history.
`ui.renderProcess`	boolean	`true`	Enable Ink render process for the UI.
`ui.terminalBuffer`	boolean	`false`	Use the new terminal buffer architecture for rendering.
`ui.useBackgroundColor`	boolean	`true`	Whether to use background colors in the UI.
`ui.incrementalRendering`	boolean	`true`	Enable incremental rendering for the UI. This option will reduce flickering but may cause rendering artifacts.
`ui.showSpinner`	boolean	`true`	Show the spinner during operations.
`ui.loadingPhrases`	enum	`"off"`	What to show while the model is working: tips, witty comments, all, or off.
`ui.errorVerbosity`	enum	`"low"`	Controls whether recoverable errors are hidden (low) or fully shown (full).
`ui.customWittyPhrases`	array	`[]`	Custom witty phrases to display during loading.
`ui.accessibility.enableLoadingPhrases`	boolean	`true`	@deprecated Use ui.loadingPhrases instead.

`ui.accessibility.screenReader`	boolean	`false`	Render output in plain-text to be more screen reader accessible.
`ide.enabled`	boolean	`false`	Enable IDE integration mode.
`ide.hasSeenNudge`	boolean	`false`	Whether the user has seen the IDE integration nudge.
`privacy.usageStatisticsEnabled`	boolean	`true`	Enable collection of usage statistics.
`billing.overageStrategy`	enum	`"ask"`	How to handle quota exhaustion when AI credits are available. 'ask' prompts each time, 'always' automatically uses...
`billing.vertexAi.requestType`	enum	`undefined`	Sets the X-Vertex-AI-LLM-Request-Type header for Vertex AI requests.
`billing.vertexAi.sharedRequestType`	enum	`undefined`	Sets the X-Vertex-AI-LLM-Shared-Request-Type header for Vertex AI requests.
`model.name`	string	`undefined`	The Gemini model to use for conversations.
`model.maxSessionTurns`	number	`-1`	Maximum number of user/model/tool turns to keep in a session. -1 means unlimited.
`model.summarizeToolOutput`	object	`undefined`	Enables or disables summarization of tool output.
`model.compressionThreshold`	number	`0.5`	The fraction of context usage at which to trigger context compression (e.g. 0.2, 0.3).
`model.disableLoopDetection`	boolean	`false`	Disable automatic detection and prevention of infinite loops.
`model.skipNextSpeakerCheck`	boolean	`true`	Skip the next speaker check.
`modelConfigs.aliases`	object		Named presets for model configs.
`modelConfigs.customAliases`	object	`{}`	Custom named presets for model configs.
`modelConfigs.customOverrides`	array	`[]`	Custom model config overrides.
`modelConfigs.overrides`	array	`[]`	Apply specific configuration overrides based on matches, with a primary key of model (or alias).
`modelConfigs.modelDefinitions`	object		Registry of model metadata, including tier, family, and features.
`modelConfigs.modelIdResolutions`	object		Rules for resolving requested model names to concrete model IDs based on context.
`modelConfigs.classifierIdResolutions`	object		Rules for resolving classifier tiers (flash, pro) to concrete model IDs.
`modelConfigs.modelChains`	object		Availability policy chains defining fallback behavior for models.
`agents.overrides`	object	`{}`	Override settings for specific agents, e.g. to disable the agent, set a custom model config, or run config.
`agents.browser.sessionMode`	enum	`"persistent"`	Session mode: 'persistent', 'isolated', or 'existing'.
`agents.browser.headless`	boolean	`false`	Run browser in headless mode.
`agents.browser.profilePath`	string	`undefined`	Path to browser profile directory for session persistence.

`agents.browser.visualModel`	string	`undefined`	Model for the visual agent's analyze_screenshot tool.
`agents.browser.allowedDomains`	array		A list of allowed domains for the browser agent (e.g., ["github.com", "*.google.com"]).
`agents.browser.disableUserInput`	boolean	`true`	Disable user input on browser window during automation.
`agents.browser.maxActionsPerTask`	number	`100`	The maximum number of tool calls allowed per browser task.
`agents.browser.confirmSensitiveActions`	boolean	`false`	Require manual confirmation for sensitive browser actions (e.g., fill_form, evaluate_script).
`agents.browser.blockFileUploads`	boolean	`false`	Hard-block file upload requests from the browser agent.
`context.fileName`	string	string[]	`undefined`
`context.importFormat`	string	`undefined`	The format to use when importing memory.
`context.includeDirectoryTree`	boolean	`true`	Whether to include the directory tree of the current working directory in the initial request to the model.
`context.discoveryMaxDirs`	number	`200`	Maximum number of directories to search for memory.
`context.memoryBoundaryMarkers`	array		File or directory names that mark the boundary for GEMINI.md discovery.
`context.includeDirectories`	array	`[]`	Additional directories to include in the workspace context.
`context.loadMemoryFromIncludeDirectories`	boolean	`false`	Controls how /memory reload loads GEMINI.md files.
`context.fileFiltering.respectGitIgnore`	boolean	`true`	Respect .gitignore files when searching.
`context.fileFiltering.respectGeminiIgnore`	boolean	`true`	Respect .geminiignore files when searching.
`context.fileFiltering.enableFileWatcher`	boolean	`false`	Enable file watcher updates for @ file suggestions (experimental).
`context.fileFiltering.enableRecursiveFileSearch`	boolean	`true`	Enable recursive file search functionality when completing @ references in the prompt.
`context.fileFiltering.enableFuzzySearch`	boolean	`true`	Enable fuzzy search when searching for files.
`context.fileFiltering.customIgnoreFilePaths`	array	`[]`	Additional ignore file paths to respect.
`tools.sandbox`	string	`undefined`	Legacy full-process sandbox execution environment.
`tools.sandboxAllowedPaths`	array	`[]`	List of additional paths that the sandbox is allowed to access.
`tools.sandboxNetworkAccess`	boolean	`false`	Whether the sandbox is allowed to access the network.

`tools.shell.enableInteractiveShell`	boolean	`true`	Use node-pty for an interactive shell experience.
`tools.shell.backgroundCompletionBehavior`	enum	`"silent"`	Controls what happens when a background shell command finishes. 'silent' (default): quietly exits in background....
`tools.shell.pager`	string	`"cat"`	The pager command to use for shell output.
`tools.shell.showColor`	boolean	`true`	Show color in shell output.
`tools.shell.inactivityTimeout`	number	`300`	The maximum time in seconds allowed without output from the shell command.
`tools.shell.enableShellOutputEfficiency`	boolean	`true`	Enable shell output efficiency optimizations for better performance.
`tools.core`	array	`undefined`	Restrict the set of built-in tools with an allowlist.
`tools.allowed`	array	`undefined`	Tool names that bypass the confirmation dialog.
`tools.confirmationRequired`	array	`undefined`	Tool names that always require user confirmation.
`tools.exclude`	array	`undefined`	Tool names to exclude from discovery.
`tools.discoveryCommand`	string	`undefined`	Command to run for tool discovery.
`tools.callCommand`	string	`undefined`	Defines a custom shell command for invoking discovered tools.
`tools.useRipgrep`	boolean	`true`	Use ripgrep for file content search instead of the fallback implementation.
`tools.truncateToolOutputThreshold`	number	`40000`	Maximum characters to show when truncating large tool outputs.
`tools.disableLLMCorrection`	boolean	`true`	Disable LLM-based error correction for edit tools.
`mcp.serverCommand`	string	`undefined`	Command to start an MCP server.
`mcp.allowed`	array	`undefined`	A list of MCP servers to allow.
`mcp.excluded`	array	`undefined`	A list of MCP servers to exclude.
`useWriteTodos`	boolean	`true`	Enable the write_todos tool.
`security.toolSandboxing`	boolean	`false`	Tool-level sandboxing.
`security.disableYoloMode`	boolean	`false`	Disable YOLO mode, even if enabled by a flag.
`security.disableAlwaysAllow`	boolean	`false`	Disable "Always allow" options in tool confirmation dialogs.
`security.enablePermanentToolApproval`	boolean	`false`	Enable the "Allow for all future sessions" option in tool confirmation dialogs.
`security.autoAddToPolicyByDefault`	boolean	`false`	When enabled, the "Allow for all future sessions" option becomes the default choice for low-risk tools in trusted...

`security.blockGitExtensions`	boolean	`false`	Blocks installing and loading extensions from Git.
`security.allowedExtensions`	array	`[]`	List of Regex patterns for allowed extensions.
`security.folderTrust.enabled`	boolean	`true`	Setting to track whether Folder trust is enabled.
`security.environmentVariableRedaction.allowed`	array	`[]`	Environment variables to always allow (bypass redaction).
`security.environmentVariableRedaction.blocked`	array	`[]`	Environment variables to always redact.
`security.environmentVariableRedaction.enabled`	boolean	`false`	Enable redaction of environment variables that may contain secrets.
`security.auth.selectedType`	string	`undefined`	The currently selected authentication type.
`security.auth.enforcedType`	string	`undefined`	The required auth type.
`security.auth.useExternal`	boolean	`undefined`	Whether to use an external authentication flow.
`security.enableConseca`	boolean	`false`	Enable the context-aware security checker.
`advanced.autoConfigureMemory`	boolean	`true`	Automatically configure Node.js memory limits.
`advanced.dnsResolutionOrder`	string	`undefined`	The DNS resolution order.
`advanced.excludedEnvVars`	array		Environment variables to exclude from project context.
`advanced.ignoreLocalEnv`	boolean	`false`	Whether to ignore generic .env files in the project directory.
`advanced.bugCommand`	object	`undefined`	Configuration for the bug report command.
`experimental.gemma`	boolean	`true`	Enable access to Gemma 4 models via Gemini API.
`experimental.voiceMode`	boolean	`false`	Enable experimental voice dictation and commands (/voice, /voice model).
`experimental.voice.activationMode`	enum	`"push-to-talk"`	How to trigger voice recording with the Space key.
`experimental.voice.backend`	enum	`"gemini-live"`	The backend to use for voice transcription.
`experimental.voice.whisperModel`	enum	`"ggml-base.en.bin"`	The Whisper model to use for local transcription.
`experimental.voice.stopGracePeriodMs`	number	`4000`	How long to wait for final transcription after stopping recording.

`experimental.adk.agentSessionNoninteractiveEnabled`	boolean	`false`	Enable non-interactive agent sessions.
`experimental.adk.agentSessionInteractiveEnabled`	boolean	`false`	Enable the agent session implementation for the interactive CLI.
`experimental.adk.agentSessionSubagentEnabled`	boolean	`false`	Route subagent invocations through the AgentSession protocol instead of legacy executors.
`experimental.enableAgents`	boolean	`true`	Enable local and remote subagents.
`experimental.worktrees`	boolean	`false`	Enable automated Git worktree management for parallel work.
`experimental.extensionManagement`	boolean	`true`	Enable extension management features.
`experimental.extensionConfig`	boolean	`true`	Enable requesting and fetching of extension settings.
`experimental.extensionRegistry`	boolean	`false`	Enable extension registry explore UI.
`experimental.extensionRegistryURI`	string		The URI (web URL or local file path) of the extension registry.
`experimental.extensionReloading`	boolean	`false`	Enables extension loading/unloading within the CLI session.
`experimental.useOSC52Paste`	boolean	`false`	Use OSC 52 for pasting.
`experimental.useOSC52Copy`	boolean	`false`	Use OSC 52 for copying.
`experimental.taskTracker`	boolean	`false`	Enable task tracker tools.
`experimental.modelSteering`	boolean	`false`	Enable model steering (user hints) to guide the model during tool execution.
`experimental.directWebFetch`	boolean	`false`	Enable web fetch behavior that bypasses LLM summarization.
`experimental.dynamicModelConfiguration`	boolean	`false`	Enable dynamic model configuration (definitions, resolutions, and chains) via settings.
`experimental.gemmaModelRouter.enabled`	boolean	`false`	Enable the Gemma Model Router (experimental).
`experimental.gemmaModelRouter.autoStartServer`	boolean	`false`	Automatically start the LiteRT-LM server when Gemini CLI starts and the Gemma router is enabled.
`experimental.gemmaModelRouter.binaryPath`	string	`""`	Custom path to the LiteRT-LM binary.
`experimental.gemmaModelRouter.classifier.host`	string		The host of the classifier.
`experimental.gemmaModelRouter.classifier.model`	string		The model to use for the classifier.
`experimental.stressTestProfile`	boolean	`false`	Significantly lowers token limits to force early garbage collection and distillation for testing purposes.
`experimental.autoMemory`	boolean	`false`	Automatically extract memory patches and skills from past sessions in the background.
`experimental.generalistProfile`	boolean	`false`	Suitable for general coding and software development tasks.

`experimental.powerUserProfile`	boolean	`false`	Less cache friendly version of the generalist profile.
`experimental.contextManagement`	boolean	`false`	Enable logic for context management.
`experimental.topicUpdateNarration`	boolean	`false`	Deprecated: Use general.topicUpdateNarration instead.
`skills.enabled`	boolean	`true`	Enable Agent Skills.
`skills.disabled`	array	`[]`	List of disabled skills.
`hooksConfig.enabled`	boolean	`true`	Canonical toggle for the hooks system.
`hooksConfig.disabled`	array	`[]`	List of hook names (commands) that should be disabled.
`hooksConfig.notifications`	boolean	`true`	Show visual indicators when hooks are executing.
`hooks.BeforeTool`	array	`[]`	Hooks that execute before tool execution.
`hooks.AfterTool`	array	`[]`	Hooks that execute after tool execution.
`hooks.BeforeAgent`	array	`[]`	Hooks that execute before agent loop starts.
`hooks.AfterAgent`	array	`[]`	Hooks that execute after agent loop completes.
`hooks.Notification`	array	`[]`	Hooks that execute on notification events (errors, warnings, info).
`hooks.SessionStart`	array	`[]`	Hooks that execute when a session starts.
`hooks.SessionEnd`	array	`[]`	Hooks that execute when a session ends.
`hooks.PreCompress`	array	`[]`	Hooks that execute before chat history compression.
`hooks.BeforeModel`	array	`[]`	Hooks that execute before LLM requests.
`hooks.AfterModel`	array	`[]`	Hooks that execute after LLM responses.
`hooks.BeforeToolSelection`	array	`[]`	Hooks that execute before tool selection.
`contextManagement.historyWindow.maxTokens`	number	`150000`	The number of tokens to allow before triggering compression.
`contextManagement.historyWindow.retainedTokens`	number	`40000`	The number of tokens to always retain.
`contextManagement.messageLimits.normalMaxTokens`	number	`2500`	The target number of tokens to budget for a normal conversation turn.

`contextManagement.messageLimits.retainedMaxTokens`	number	`12000`	The maximum number of tokens a single conversation turn can consume before truncation.
`contextManagement.messageLimits.normalizationHeadRatio`	number	`0.25`	The ratio of tokens to retain from the beginning of a truncated message (0.0 to 1.0).
`contextManagement.tools.distillation.maxOutputTokens`	number	`10000`	Maximum tokens to show to the model when truncating large tool outputs.
`contextManagement.tools.outputMasking.protectLatestTurn`	boolean	`true`	Ensures the absolute latest turn is never masked, regardless of token count.
`admin.secureModeEnabled`	boolean	`false`	If true, disallows YOLO mode and “Always allow” options from being used.
`admin.extensions.enabled`	boolean	`true`	If false, disallows extensions from being installed or used.
`admin.mcp.enabled`	boolean	`true`	If false, disallows MCP servers from being used.
`admin.mcp.config`	object	`{}`	Admin-configured MCP servers (allowlist).
`admin.mcp.requiredConfig`	object	`{}`	Admin-required MCP servers that are always injected.
`admin.skills.enabled`	boolean	`true`	If false, disallows agent skills from being used.
`GOOGLE_APPLICATION_CREDENTIALS`	string		The path to your Google Application Credentials JSON file.
`SEATBELT_PROFILE`	macOS specific		.

Your Autonomy Setup Checklist

- [ ] Agents run in a dev container, VM, cloud box, or separate user account
- [ ] Separate GitHub account or app for agents; fine-grained token with full repo permissions
- [ ] Hard spend limits set on every AI and cloud account
- [ ] Everything in Git; `main` protected by status checks; auto-merge allowed
- [ ] `~/.claude/settings.json`: `defaultMode` set to `auto` or `bypassPermissions`, plus `skipDangerousModePermissionPrompt`
- [ ] Claude Desktop: **Allow bypass permissions mode** on; Cowork: **Automatically approve**
- [ ] Every connector tool set to **Always allow**
- [ ] Cursor: **Run Everything** (or Auto-review with allow instructions)
- [ ] VS Code: `chat.tools.global.autoApprove` on, or Autopilot
- [ ] Codex `config.toml`: `approval_policy = "never"`, `sandbox_mode = "danger-full-access"`; Gemini alias with `--yolo`
- [ ] n8n: API key, instance-level MCP on, workflows enabled for MCP; `NODES_EXCLUDE="[]"` if self-hosted
- [ ] Notion: MCP connected; API connection with all capabilities and top-level pages shared
- [ ] Unattended runs have `--max-turns` and `--max-budget-usd` limits

Glossary

Allowlist	A list of pre-approved actions an agent may take without asking.
Auto mode	Claude Code's permission mode in which a classifier reviews actions instead of a person.
Bypass permissions	A mode that skips approval prompts entirely (`--dangerously-skip-permissions` in Claude Code).
Classifier	A second AI model that judges whether an agent's action is safe and matches the request.
Containment	Limiting what an agent can reach (a container, separate accounts, budgets) instead of limiting what it may do.
Critical path	A folder, such as the root, your home folder, or the working directory, that Claude Code protects from deletion in every mode.
Dev container	A Docker container configured as a development environment, used to isolate agents.
Fine-grained token	A GitHub access token limited to chosen repositories and permissions.
Headless / print mode	Running an agent non-interactively from a script, such as `claude -p`.
Prompt injection	Instructions hidden in content an agent reads, meant to hijack its behavior.
Ruleset	GitHub settings that protect branches, with an optional bypass list.
Sandbox	An isolated environment limiting the files and network a command can reach.
Sudo mode	GitHub's re-authentication step for sensitive actions in the web interface.
YOLO mode	Informal name for fully automatic approval in tools like Gemini CLI, Codex, and Copilot CLI.